

Swinburne University of Technology

School of Science, Computing and Emerging Technologies

ASSIGNMENT AND PROJECT COVER SHEET

Subject Code: SWE30003

Unit Title: Software Architectures and Design

Assignment number and title: 2, Object Design

Due date: 11:59pm, 10th May 2026

Tutorial Day and time: Wednesday 4:30PM

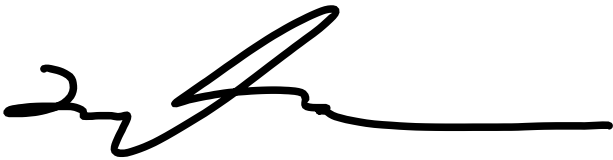
Project Group: 3

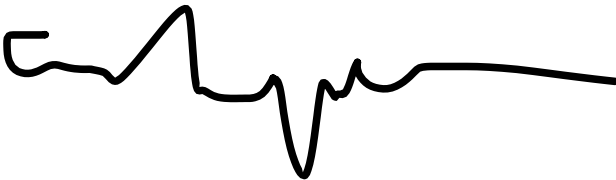
Lecturer: Dr Mandeep Dhindsa

Tutor: Dr Mohammad Abuhassan

To be completed as this is a group assignment

We declare that this is a group assignment and that no part of this submission has been copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part been written for us by another person.

ID Number	Name	Signature
<u>104565600</u>	<u>Jeremy Allan</u>	

<u>104093910</u>	<u>Pulkit Pannu</u>	
<u>101614596</u>	<u>Justin Chen</u>	
<u>103986022</u>	<u>Christom Alappadan</u>	

Marker's comments:

Total Mark:

SWE30003 - Assignment 2: Object Design

Favourite Books Online Bookstore System

Wednesday, 4:30pm, Group 3

Jeremy Allan

Pulkit Pannu

Justin Chen

Christom Alappadan

May 10, 2026

Contents

1	Executive Summary & Introduction.....	7
1.1	Executive Summary	7
1.2	Problem Overview	7
1.3	Outlook of Solution	7
2	Problem Analysis	9
2.1	Assumptions	9
2.2	Simplifications	9
2.3	Design Justification	10
2.4	Discarded Class List	10
3	Candidate Classes.....	12
3.1	Candidate Class List	12
3.2	UML Class Diagram.....	13
3.3	CRC Cards	15
3.3.1	Item	15
3.3.2	Book	15
3.3.3	Catalogue.....	16
3.3.4	ShoppingCart.....	16
3.3.5	Order	17
3.3.6	PaymentMethod.....	17
3.3.7	Card	18
3.3.8	PayPal	18
3.3.9	Courier.....	18

3.3.10	Address (Data-Holder).....	19
3.3.11	User.....	19
3.3.12	Customer.....	19
3.3.13	Admin.....	20
3.3.14	Guest.....	20
3.3.15	Invoice.....	20
3.3.16	SalesReport	21
3.3.17	DatabaseConnector.....	21
3.3.18	ShippingManager	21
3.3.19	SalesReportGenerator.....	22
3.3.20	BookstoreSystem.....	22
4	Quality of Design	23
4.1	Design Heuristics	23
4.2	Design Patterns.....	24
4.2.1	Creational Patterns:.....	24
4.2.2	Behavioural Pattern:	25
4.2.3	Structural Patterns:.....	26
5	Illustration of Bootstrap Process	27
6	Basic Verification (Use Scenarios).....	28
6.1	Scenario 1: Customer Checks out Shopping Cart	28
6.2	Scenario 2: Guest Makes a Customer Account.....	29
6.3	Scenario 3: Admin Generates a Sales Report	30
6.4	Scenario 4: Admin Dispatches an order for Shipment.....	31

6.5	Customer Browses Book Catalogue and adds to Cart (Upon Application Start).....	32
7	References	33
8	Appendices	34
8.1	Appendix A: Assignment 1 Submission	34
8.2	Appendix B: Signed Cover Sheet	34
8.3	Appendix C: Contribution Collaboration Document.....	34

1 Executive Summary & Introduction

1.1 Executive Summary

This document provides the initial object-oriented design for the Favourite Books online store. This design builds on top of the initial specification phase which identified and defined the functional requirements of the system and plans to build a structural class model.

The report defines the candidate classes and their roles and responsibilities through Class-Responsibility-Collaborator (CRC) cards. We provide a detail boot-strap initialization sequence and map out some use case scenarios to validate and verify the structural integrity of the system. Additionally, a detailed description of the design justification along with assumptions and constraints are provided in this document.

1.2 Problem Overview

The Favourite Books bookstore is currently operating only as a physical store on Glenferrie Road, which limits how many customers it can reach, ultimately capping its revenue potential. To grow, the business needs a way to expand and reach into the wider Australian market via an online presence.

The main challenge is creating a system that allows customers to access the store at any time and from anywhere, without interfering with how things currently run behind the scenes. Customers should be able to browse the catalogue and manage their shopping carts on their own, whenever it suits them. At the same time, the system should make it easier for the users to process payments and conveniently track their orders and invoices. Additionally, the system should clearly provide a way for the staff to handle order confirmations, inventory management, statistics and delivery/shipment management.

1.3 Outlook of Solution

The solution is designed carefully with the Favourite Bookstore's requirements in mind and accounts for future extensibility due to its modular nature, enabling the business to easily expand its online products and services over time. The architecture utilizes a central controller responsible for routing specific tasks to various entities and processors.

The system will provide customers to browse the book catalogue, login or create accounts, manage their shopping cart and securely process payments. It also integrates a dedicated management interface for the bookstore staff and admins to easily update, add and configure products (books), manage the overall catalogue and inventory, shipping and business analytics.

The proposed solution aims to incorporate a relational database to act as a secure, central hub for all the bookstore's information. This will safely store and manage everything including customer accounts, order histories, inventory and ensuring the data is always accurate and easy for the system to retrieve.

2 Problem Analysis

The main pain points for the Favourite Bookstore owners were their limited customer reach being a physical-only store and a lack of automated sales analytics, making it difficult to oversee business operations effectively. The proposed system design aims to solve these issues by creating a secure and scalable e-commerce platform to increase accessibility and availability to customers and handle internal reporting.

2.1 Assumptions

1. All payment processing will be handled by a third-party payment provider.
2. All financial transactions will be processed in a single currency for now.
3. Delivery and shipment will be handled by external courier services.
4. External payment gateways and third-party shipment tracking APIs are assumed to be reliable and provide continuous uptime.
5. The system will be hosted on cloud services.
6. Customers will have access to internet and internet-enabled devices.
7. Customers will have basic digital literacy.
8. The physical inventory is managed by the staff and not the system. The system assumes that the inventory data is accurate.
9. Only item in the catalogue are books.
10. The data entered during a guest checkout session is not stored as a persistent, searchable user profile. It is only retained for the lifecycle of the specific order.
11. The system uses a single, centralized relational database which will handle and store all the data without the need for distributed database syncing.
12. Email will be the sole method for sending invoices, receipts and delivery notifications to customers.
13. The bookstore already has all the required necessary wholesale agreements with book distributors or publishers.

2.2 Simplifications

Based on the assumptions made to the requirements and to ensure that the project remains focused on the core functionalities and requirements, some simplifications were made to the system's scope:

- Loyalty Programs and Rent-before-buying features are out of scope.

- The system currently doesn't handle refunds and returns, and it will be done manually by the staff.
- Advanced recommendation and marketing algorithms are out of scope.
- Discounts and promotions are out of scope.
- The system won't be responsible for managing physical delivery logistics.
- No pickup option available. Users can only deliver their books when using the online platform.
- Partial shipments or split backorders are not supported.
- E-books and digital equivalents are excluded from the catalogue.
- Only first-hand books will be part of the catalogue.
- The system does not manage stock purchasing.

2.3 Design Justification

The chosen design follows the object-oriented best practices using a Responsibility Driven Design approach, allowing the system to be modular and loosely coupled. The entities and processors follow the Single Responsibility Principle (SRP) making it easier to maintain and extend as the business logic grows.

By default, the system is designed to be backed up by a relational database. The class lists defined will act as an interface to the database connector which is a singleton used by each entity. It is assumed that the database is optimally indexed and normalized and doesn't contain any redundant information.

The design uses abstraction to prevent the main controllers and manager classes from turning into monolithic classes. Base classes like Item and User provides future extensibility to the system if the business decides to add more online products or user profiles.

The system was designed while keeping several design patterns in mind, such as Factory method pattern in the User base class, Strategy patterns for handling payments and transactions and Singleton pattern for preventing resource conflicts and data consistency.

2.4 Discarded Class List

After the initial design phase, several candidate classes were discarded due to design simplifications and out of scope elements.

- RefundController: Since refunds and returns are out of scope, this class was dropped.

- UserFactory: Instead of having a factory class we decided it would be better to use the factory method design pattern in the base class.
- RecommendationEngine: To simplify the browsing experience the advanced recommendation algorithms were decided to be out of scope for this project.
- Payment: We decided to remove the Payment class as it was an unnecessary layer, and transactions would be processed by external payment gateways.

3 Candidate Classes

3.1 Candidate Class List

Entities:
• Item (Abstract)
○ Book
• Catalogue
• ShoppingCart
• Order
• PaymentMethod (Abstract)
○ Card
○ PayPal
• Courier
• Address (Data-Holder)
• User: (Abstract)
○ Customer
○ Admin
○ Guest
• Invoice (Data-Holder)
• SalesReport
• Database Connector
• ShippingManager
Generators/Processors:
• SalesReportGenerator
Controller/Coordinator Classes:
• BookstoreSystem

3.2 UML Class Diagram

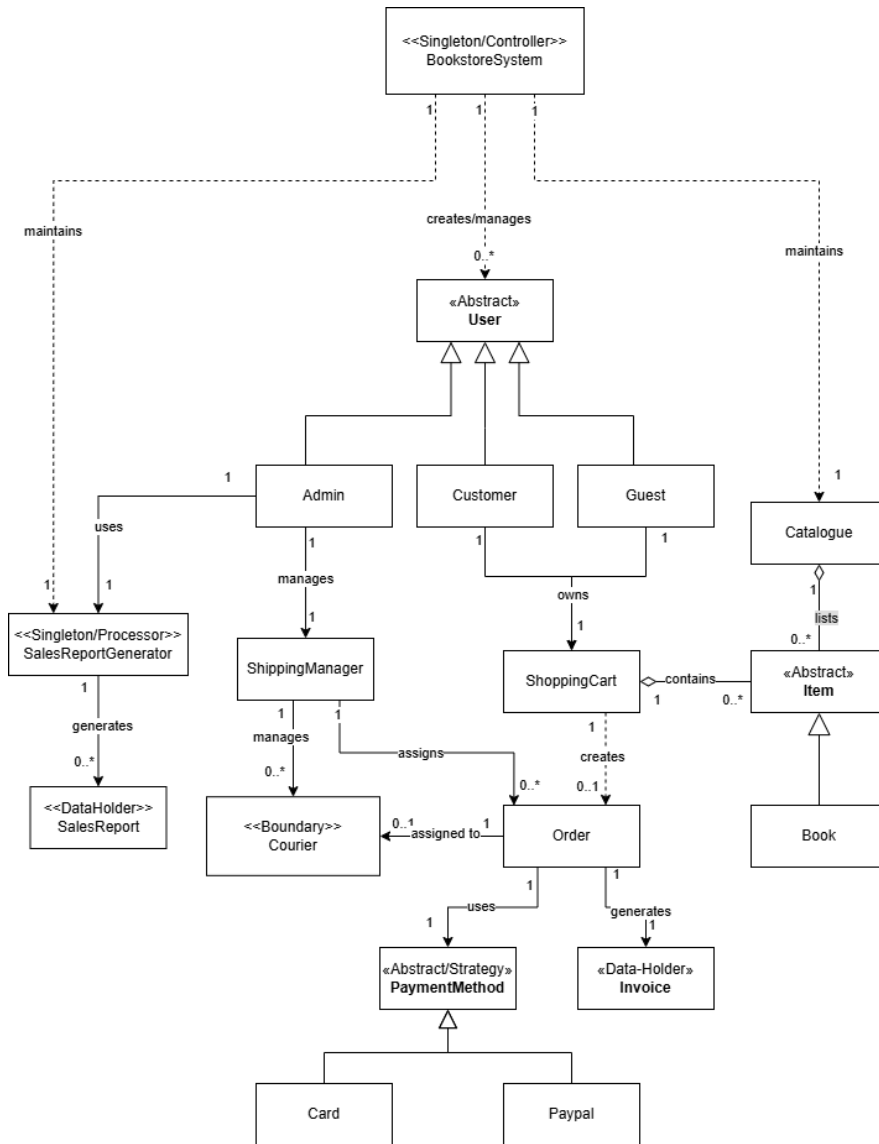


Figure 1: UML Class Diagram

This UML diagram encompasses the initial object-oriented design for the Favourite Books Online Bookstore system. It focuses on candidate classes and their relationships, rather than detailed attributes or method signatures.

The main controller is the BookstoreSystem, marked as a singleton/Controller as there should only be one main system controller coordinating the overall book system. It manages major system objects such as Users, Catalogue and SalesReportGenerator.

Admin, Customer, and Guest inherit from User because they are each specialised types of Users. Customers and Guests do have ShoppingCart's each, but Guests will have limited functionality.

The Catalogue manages the available items that can be browsed or purchased. The one and only Item type available is the Book at the moment, but the abstract Item class allows the system to support different item types in the future, such as eBook, accessories, audio books.

Coming back to the ShoppingCart, it contains selected Items by the User before checkout. Once the User does proceed through checkout, an Order class is created, copying the Items from the ShoppingCart (if successful).

The Order is the confirmed purchase after the checkout. It collaborates with PaymentMethod, Invoice and Courier as in, the Order will have these other methods attached to them. For example, the Order will have an Invoice generated, a certain type of PaymentMethod and a Courier assigned for it.

In regard to PaymentMethod, it is an Abstract class, showing the use of the Strategy Pattern for payment processing. Both Card and PayPal inherit the PaymentMethod.

On the Admin side of things, the design separates the main administrative tasks into classes such as ShippingManager & SalesReportGenerator. This prevents the Admin class from being overloaded and supports a cleaner separation of concerns.

The ShippingManager is responsible for managing shipment-related responsibilities once an order has been created. The main one being the assignment of Courier to an Order for delivery. The Courier class is marked as a boundary class because it represents interaction with an external delivery service.

The other administrative function is the SalesReportGenerator which generates SalesReport. The SalesReport class is marked as a data-holder, as it stores the result of generated sales statistics.

The design is not final, intentionally high-level and can be refined further during detailed design or implementation.

3.3 CRC Cards

3.3.1 Item

Class Name: Item	
Super Class: -	
Description: An abstract base class that represents a generic product in the Catalogue. It states the foundational attributes for all items that can be purchased and listed in the Catalogue and added to the ShoppingCart.	
Responsibilities	Collaborators
Knows metadata (id, price)	NA
Maintains current inventory stock levels	Database Connector

3.3.2 Book

Class Name: Book	
Super Class: - Item	
Description: A child class implementation of the Item parent class which wraps and encapsulates the specific metadata for a book and serves as the primary purchasable product managed by the Catalogue	
Responsibilities	Collaborators
Knows book attributes (author, publisher, genre)	NA

3.3.3 Catalogue

Class Name: Catalogue	
Super Class: -	
Description: A central manager component maintained by the BookstoreSystem controller which acts as the definitive inventory list and provides all the available abstract Item entities and interfaces with the ShoppingCart	
Responsibilities	Collaborators
Manages the list of available Items objects	Items
Performs searches and filters Items by genre or title	Item, Database Connector
Enables user to add Item to ShoppingCart	Item, ShoppingCart

3.3.4 ShoppingCart

Class Name: ShoppingCart	
Super Class: -	
Description: A transactional container that is owned by either a Customer or a Guest. It aggregates all the selected Items from the Catalogue and handles the transition from browsing to checkout by creating a new Order	
Responsibilities	Collaborators
Maintains a temporary list of selected books and quantities	Book
Calculates subtotal of items currently in the cart	Book
Initiates checkout and instantiates a new Order	Order, Customer, PaymentMethod

3.3.5 Order

Class Name: Order	
Super Class: -	
Description: It is the core entity created by the ShoppingCart for transactional purposes. Once it is initialized it uses a PaymentMethod Strategy for processing payments, then generates an Invoice and is assigned a Courier Boundary object by the ShippingManager	
Responsibilities	Collaborators
Knows Order id, date, price, and user details	Customer, ShoppingCart, Database Connector
Maintains reference to purchased items and total price	Book
Records shipping address and selected courier	Address, Courier
Instantiates a new Invoice upon successful payment confirmation	Invoice
Tracks payment and fulfillment status	Payment, ShippingManager

3.3.6 PaymentMethod

Class Name: PaymentMethod	
Super Class: -	
Description: An Abstract base class acting as the Strategy Pattern interface for processing payments. It decouples the Order from specific payment gateways and allows for interchangeable use of its concrete methods such as Card or PayPal	
Responsibilities	Collaborators
Defines standard format for authorizing transactions and withdrawing a certain amount from third party API	NA
Reports transaction success or failure back to the Order	Order

3.3.7 Card

Class Name: Card	
Super Class: PaymentMethod	
Description: A concrete Strategy implementation of the PaymentMethod interface. It is used by an Order object to securely authorize transactions via external payment gateways	
Responsibilities	Collaborators
• Authorizes transaction amount via external financial API	NA

3.3.8 PayPal

Class Name: PayPal	
Super Class: PaymentMethod	
Description: A concrete Strategy implementation of the PaymentMethod interface. It is used by an Order object to securely authorize transactions via external payment gateways	
Responsibilities	Collaborators
• Authorizes transaction amount via external financial API	NA

3.3.9 Courier

Class Name: Courier	
Super Class: -	
Description: A Boundary class managed by the ShippingManager and assigned to an Order class object. It wraps interactions with external delivery services to handles the dispatch and shipment tracking abilities	
Responsibilities	Collaborators
• Knows company details and trackingAPI	NA
• Provides tracking updates for a specific OrderID	Order

3.3.10 Address (Data-Holder)

Class Name: Address	
Super Class: -	
Description: A simple Data-Holder class defining the physical address of a user. storing It is utilized mainly by Order and User classes to describe a standard shipping destination that can be used and accessed with ease.	
Responsibilities	Collaborators
Knows street number, name, and suburb details	NA

3.3.11 User

Class Name: User	
Super Class: -	
Description: Abstract base class managed by the BookstoreSystem class. It defines the common authentication data and uses a Factory Method to dynamically initialize and instantiate specific user profiles, like Admin, Customer or Guest	
Responsibilities	Collaborators
Stores unique user account and authentication data	Database Connector
Creates specific Customer, Admin, or Guest users using UserFactory method	

3.3.12 Customer

Class Name: Customer	
Super Class: User	
Description: A child class inherited from the User parent class representing a registered shopper/customer. It exclusively has its own persistent ShoppingCart to select and add items and controls the creation of confirmed Orders	
Responsibilities	Collaborators
Holds personal information	Database Connector
Maintains active ShoppingCart	ShoppingCart
Views order history and tracking status	Order, Courier

3.3.13 Admin

Class Name: Admin	
Super Class: User	
Description: A child class inherited from the User parent class representing an internal staff member. It serves as an administrative user that has access to manage the shipping subsystem and use the SalesReportGenerator for getting business statistics details	
Responsibilities	Collaborators
Updates book data and catalogue availability	Catalogue, Book
Requests business performance analytics	SalesReportGenerator
Views shipping/packaging information	ShippingManager

3.3.14 Guest

Class Name: Guest	
Super Class: User	
Description: A child class inherited from the User parent class representing an unregistered online shopper. It owns a temporary ShoppingCart for a single-session checkout without storing user data/credentials.	
Responsibilities	Collaborators
Enables one-time purchases without persistent credentials	ShoppingCart, Order

3.3.15 Invoice

Class Name: Invoice	
Super Class: -	
Description: A Data-Holder class generated directly by an Order class after a successful checkout. It holds, wraps and formats the final tax details of the transaction for customer records	
Responsibilities	Collaborators
Formats and presents tax/purchase details from an Order	Order

3.3.16 SalesReport

Class Name: SalesReport	
Super Class: -	
Description: A Data-Holder class particularly initialized by the SalesReportGenerator. It acts as a static container for the calculated statistics of the business requested by an Admin User	
Responsibilities	Collaborators
Knows report range (dateFrom, dateEnd)	NA
Holds statistical reportData for display	NA

3.3.17 DatabaseConnector

Class Name: Database Connector	
Super Class: -	
Description: A global Singleton class that serves as the single point of truth for data querying and persistence. It provides a standard interface for all entities to securely execute queries and retrieve stored states.	
Responsibilities	Collaborators
Executes queries to retrieve/save object states	NA

3.3.18 ShippingManager

Class Name: ShippingManager	
Super Class: -	
Description: A subsystem manager that is operated by the Admin users. It is responsible for overseeing the complete lifecycle of an order by talking confirmed Orders and assigning appropriate Courier boundary objects for physical shipment	
Responsibilities	Collaborators
Flags orders for packaging and verifies inventory	Order, Book
Assigns packages to Courier for dispatch	Courier, Order

3.3.19 SalesReportGenerator

Class Name: SalesReportGenerator	
Super Class: -	
Description: A Singleton Processor utilized especially for Admin users. It abstracts the complex logic of querying the database records to generate and populate the SalesReport data holder objects	
Responsibilities	Collaborators
• Queries order data	Order, Database Connector
• Instantiates and populates a SalesReport	SalesReport

3.3.20 BookstoreSystem

Class Name: BookstoreSystem	
Super Class: -	
Description: The central Singleton Controller for the application. It is responsible for orchestrating the system initialization, high-level components like the Catalogue, Shipping Manager, User and Sales Report Generator	
Responsibilities	Collaborators
Coordinates system bootstrap and • initialization	All Entity Classes
• Routes task requests to specific managers and generators	User, Catalogue, Shipping-Manager, SalesReportGenerator

4 Quality of Design

4.1 Design Heuristics

H1: High Cohesion: Each class is designed to capture exactly one key abstraction. For example, the Catalogue class is solely responsible for managing and searching the book collection, while the ShippingManager handles the logistics of dispatching orders.

H2: Low Coupling: Relationships between classes are minimized to reduce dependency. The ShoppingCart initiates a checkout by interacting with the abstract PaymentMethod rather than concrete implementations, ensuring that changes to a specific payment provider do not break the cart logic.

H3: Avoidance of "God Classes": To prevent the BookstoreSystem from becoming a monolithic controller, responsibilities are delegated to specific managers. The BookstoreSystem only routes high-level requests, while logic remains encapsulated within the relevant entity or processor classes.

H4: Explicit Data-Holders: Address, Invoice and SalesReport are explicitly defined as data-holder classes. These classes serve as data structure definitions rather than active behavioural objects.

H5: Base classes should be declared abstract where they represent a general type rather than a concrete object. For example, User is abstract because the system should create Customer, Admin, or Guest objects rather than a generic User object.

H6: Abstract classes should only be used where they provide a meaningful shared role. For example, PaymentMethod is used as the shared abstraction for Card and PayPal payment types.

H7: Employ consistency in language, diagrams, and responsibilities. Class names such as ShoppingCart, SalesReportGenerator, ShippingManager, and DatabaseConnector are used consistently across the UML diagram, CRC cards, design patterns, and bootstrap process.

H8: Avoid unnecessary classes where a responsibility can be handled by an existing class. For example, checkout is handled through ShoppingCart and Order rather than introducing an extra CheckoutController.

4.2 Design Patterns

4.2.1 Creational Patterns:

- **Factory Method Pattern:** The factory method pattern is used in the “User” class. This method defines the logic that is required to initialise the different users, specifically “Customer”, “Admin” or “Guest”, based on the input context. Each child/subclass of the User class will implement this method and passes their own specific implementation details to handle specific initialization. This ensures that the system can correctly generate user profiles without the User class to manage everything.

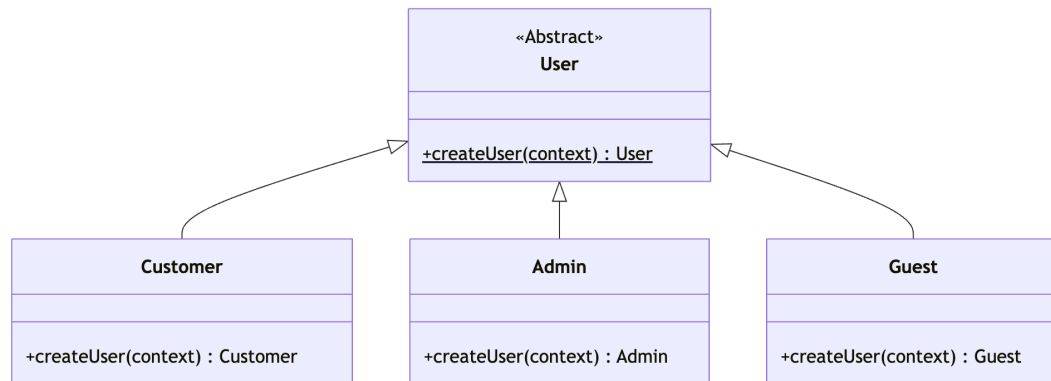


Figure 2: Factory Method Pattern Example

- **Singleton Design Pattern:** The “DatabaseConnector” is implemented as a Singleton. This ensures that only one instance of the database connection exists within the system, preventing resource conflicts and ensuring a single point of truth for data persistence. Additionally, both the central “BookstoreSystem” controller and the “SalesReportGenerator” that utilize the Singleton pattern to ensure consistency and avoid the risk of duplicated initialisations.

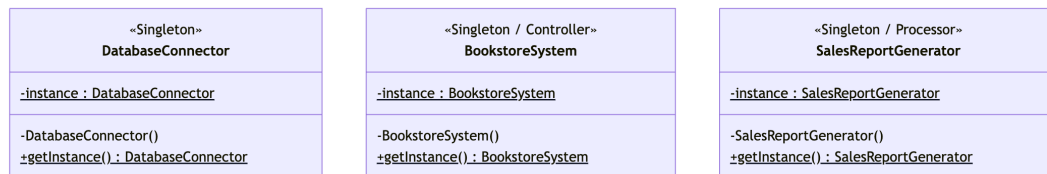


Figure 3: Singleton Design Pattern Example

4.2.2 Behavioural Pattern:

- **Strategy Design Pattern:** This pattern is applied to the “PaymentMethod” hierarchy. By defining an abstract “PaymentMethod” interface, the system can dynamically switch between different concrete strategies (e.g., Card, PayPal) at runtime. This decouples the “Order” class from the specific APIs and ensures that the system is extensible for future payments methods

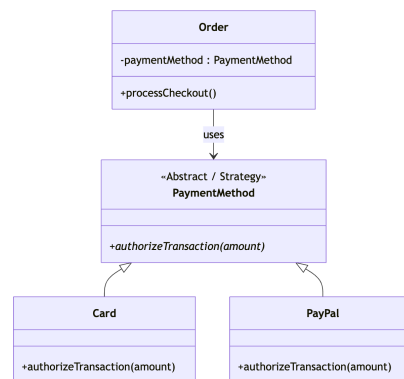


Figure 4: Strategy Design Pattern Example

4.2.3 Structural Patterns:

- **Facade / Controller Pattern:** The “BookstoreSystem” class acts as a Facade or a central Controller for the entire application. Instead of having the user interface or external actors interact directly with the internal subsystem, everything gets routed through the “BookstoreSystem” Controller. This class delegates tasks and actions to other Manager classes, such as “ShippingManager”, reducing coupling across the system and decreasing complexities.

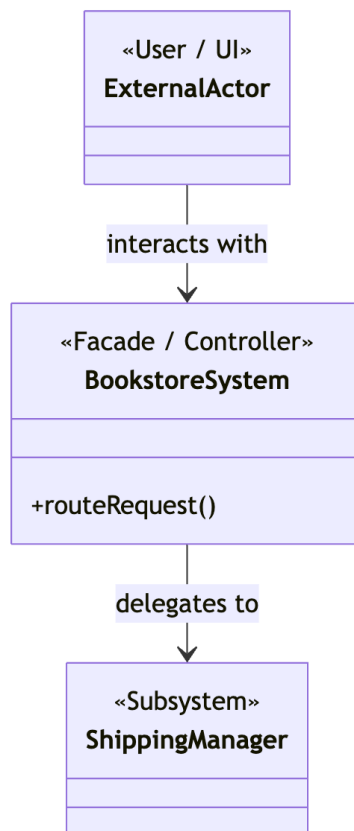


Figure 5: Facade/Controller Pattern Example

5 Illustration of Bootstrap Process

This bootstrapping process reflects a user opening the web application and placing an order.

- The bootstrap process begins when Main creates an instance of BookstoreSystem. BookstoreSystem is responsible for initialising the main objects required for the Favourite Books Online Bookstore System to operate.
- It first creates and accesses the shared DatabaseConnector, which provides access to stored information such as books, users, orders and reports. Once the database connection is available, BookstoreSystem creates the Catalogue, which retrieves the available Book objects from the DatabaseConnector.
- BookstoreSystem then creates the supporting system classes, including User, ShippingManager, Catalogue and SalesReportGenerator.
- When a user begins a session, the User class uses a Factory method to create the appropriate User object, which may be a Customer, Admin or Guest. If the user is a Customer or Guest, BookstoreSystem creates a ShoppingCart for that user. The user can then browse the Catalogue and add selected Book objects to the ShoppingCart.
- When checkout begins, ShoppingCart creates an Order using the selected Book objects, the current User, and the user's shipping Address. The Order is then associated with the selected PaymentMethod, which may be either Card or PayPal.
- Once the payment is confirmed, the Order creates an Invoice. The completed Order is then passed to ShippingManager, which assigns a Courier where delivery tracking is required.

This process ensures that objects are created in a logical order, beginning with the main system controller and then moving through catalogue access, user creation, shopping cart creation, order processing, payment, invoice generation and shipping.

6 Basic Verification (Use Scenarios)

6.1 Scenario 1: Customer Checks out Shopping Cart

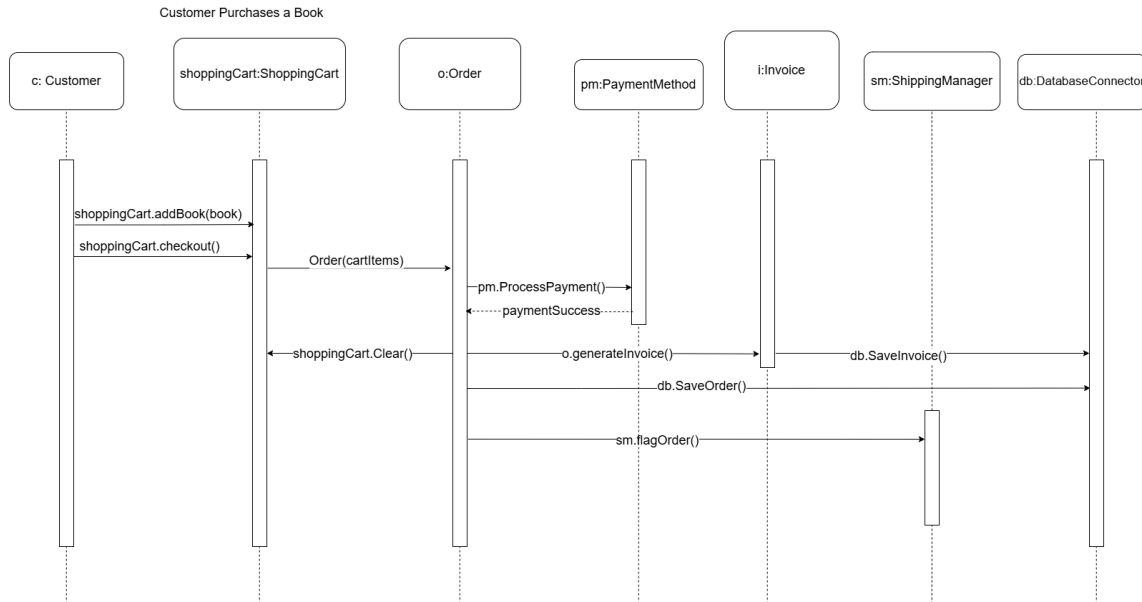


Figure 6: Use Scenario 1

This scenario verifies the core customer purchase flow. A registered customer selects books from the catalogue, confirms the shopping cart, enters delivery information, chooses a payment method and completes the order. This connects the main customer facing functions including catalogue browsing, cart management, order creation, payment processing, invoice generation and shipping preparation.

6.2 Scenario 2: Guest Makes a Customer Account

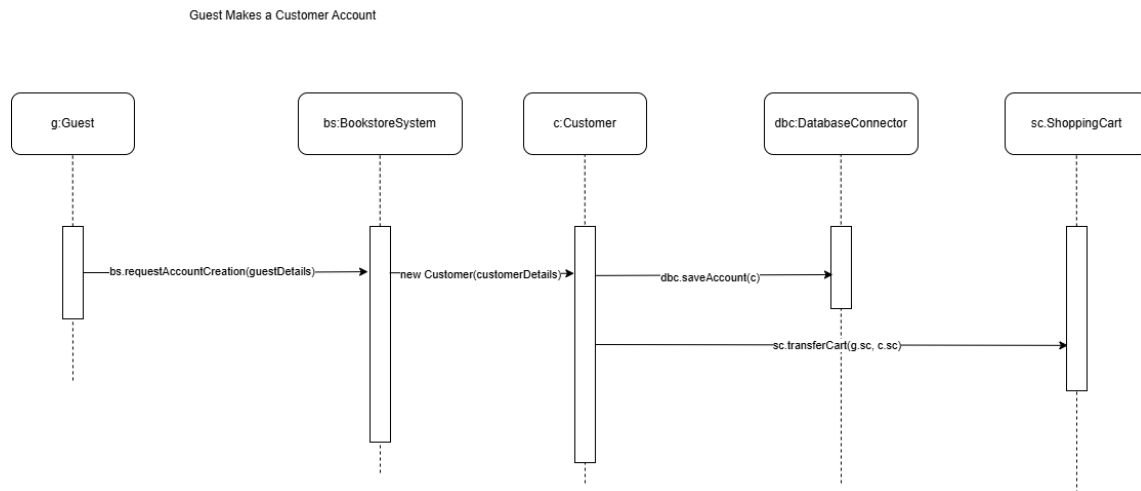


Figure 7: Use Scenario 2

This section verifies how the system handles a temporary guest becoming a persistent customer. A guest may initially browse or purchase without a permanent profile, but can later create an account so future orders, delivery details and purchase history can be linked to reusable customer identity.

6.3 Scenario 3: Admin Generates a Sales Report

Admin Generates a Sales Report

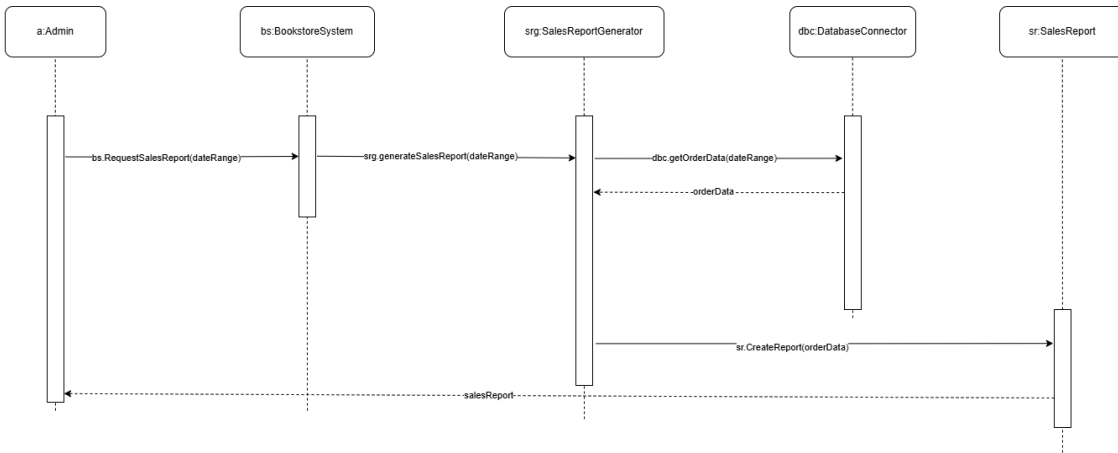


Figure 8: Use Scenario 3

This scenario verifies the reporting side of the system. An admin user requests a sales report for a selected date range, and the system retrieves order data, calculates business statistics and presents the result in a structured report object. This supports the business need for clearer internal sales analytics.

6.4 Scenario 4: Admin Dispatches an order for Shipment

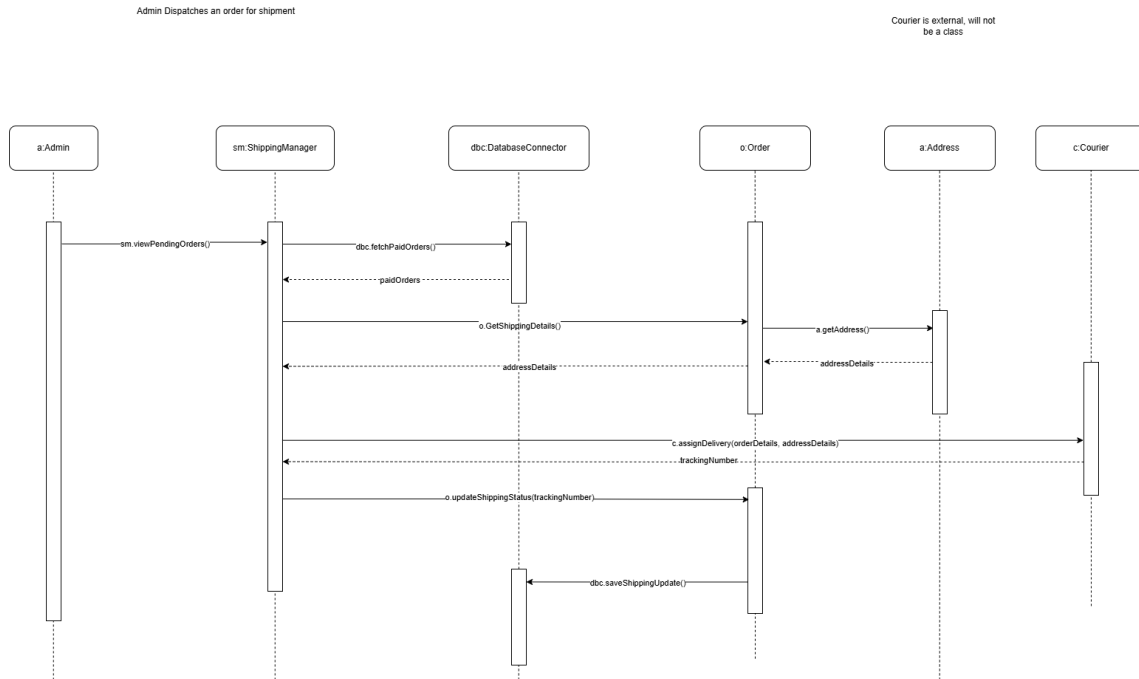


Figure 9: Use Scenario 4

This scenario verifies the order fulfilment process after payment has been completed. A staff/admin user prepares a confirmed order for dispatch, checks the required books, assigns the order to a courier and updates the shipment status. This ensures the design supports the operational side of the online bookstore.

6.5 Customer Browses Book Catalogue and adds to Cart (Upon Application Start)

Customer Browses Book Catalogue and adds to Cart

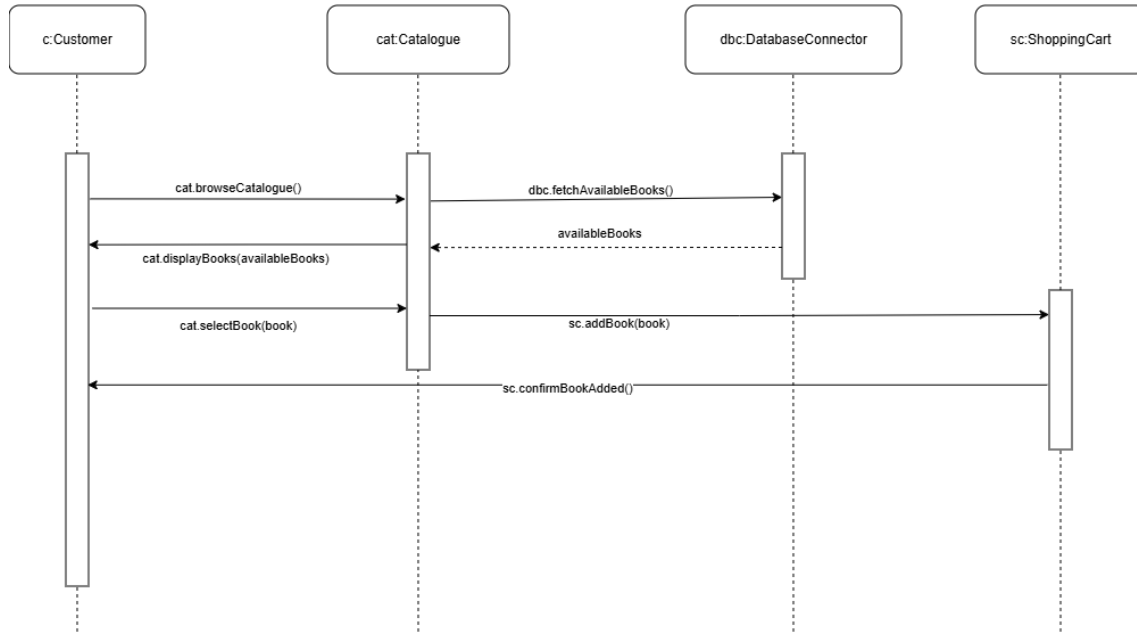


Figure 10: Use Scenario 5

In this scenario, the customer browses the online book catalogue to view the books currently available for purchase. The Catalogue retrieves the available book information from the DatabaseConnector and displays the results to the Customer. Once the customer chooses a book, the selected book is added to the customer's ShoppingCart. The ShoppingCart then confirms that the book has been successfully added, displaying output to the Customer and allowing the customer to continue browsing or proceed to checkout.

7 References

- [1] Bass, L., Clements, P., & Kazman, R. (2013). *Software Architecture in Practice* (3rd ed.). Addison-Wesley.
- [2] Dipl, U., & Gall, T. (n.d.). *Object-Oriented Design Heuristics*. Retrieved May 15, 2026, from <https://dsg.tuwien.ac.at/Teaching/Courses/SWV/slides06/reuse-04.pdf>
- [3] GeeksforGeeks. (2017, October 27). *Sequence Diagrams Unified Modeling Language (UML)*. GeeksforGeeks. <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-sequence-diagrams>
- [4] GeeksforGeeks. (2024, April 10). *ClassResponsibilityCollaboration Card*. GeeksforGeeks. <https://www.geeksforgeeks.org/system-design/class-responsibility-collaboration-card/>
- [5] *Head First Design Patterns [Book]*. (n.d.). Www.oreilly.com. <https://learning.oreilly.com/library/view/head-first-design/0596007124/>
- [6] Martin, R. C. (2004). *UML for Java programmers*. Pearson Education.
- [7] *Software Patterns, Pattern Languages, and Frameworks for Middleware and Applications*. (2024). Vanderbilt.edu. <https://www.dre.vanderbilt.edu/~schmidt/patterns-frameworks.html>
- [8] *UML Diagrams: 14 Types and Templates*. (n.d.). Gliffy by Perforce. <https://www.gliffy.com/blog/uml-2-5-diagram-types-and-templates>
- [9] w3sDesign. (2017, October). *The GoF Design Patterns Memory - Learning Object-Oriented Design & Programming*. W3sDesign. <https://w3sdesign.com/?gr=b09&ugr=proble>

8 Appendices

8.1 Appendix C: Contribution Collaboration Document

8.2 Appendix A: Assignment 1 Submission

8.3 Appendix B: Signed Cover Sheet (attached at the top of the document)

Swinburne University of Technology
School of science, Computing and Emerging Technologies

Contribution and Collaboration Document

Subject Code: SWE30003

Unit Title: Software Architectures and Design

Assignment number and title: 2, Object Design

Due date: 10th May 2026

Tutorial Day and Time: Wednesday 4:30pm

Project Group: 3

Tutor: Dr Mohammad Abuhassan (mabuhassan@swin.edu.au)

1. **Time spent** (*hours*): fill in the time spent on each significant work item in the following table.

Work Item	Member 1 Name: Justin Chen	Member 2 Name: Jeremy Allan	Member 3 Name:	Member 4 Name: Christom
Project Introduction and Task Delegation	1 hour	1 hour	1 hour	1 hour
Executive Summary & Intro			1.5 hours	
Problem Analysis			1.5 hours	
Candidate Class List	1 hour (assisted)	2 hours	2 hours (assisted / reviewed)	
UML Class Diagram	5	1 hour (assisted with diagram structure)		
CRC Cards		4 hours	1.5 hours (assisted / reviewed)	

Quality of Design		1 hour (assisted)	3 hours (updated / reviewed)	
Bootstrap Process			30 mins (updated / reviewed)	3 hours
Basic Verification (User Scenarios)	3			3 hours
Final submission and formatting		2 hours		
Total	10 hours	11 hours	11 hours	7 Hours

Note: Please adjust and add more rows, as necessary.

Description of contributions (each member provides a brief description of their contributions):

Member 1 Name: Justin Chen

- Completed Section 3.2 UML Class Diagram. Designed the UML Diagram structure using draw.io, including the object-oriented relationships between entities, multiplicities and supporting classes.
- Assisted with Section 3.1 Candidate class List by reviewing & discussing selected classes.
- Completed Section 6 Basic Verification / User Scenarios by designing sequence diagrams and descriptions used to verify customer and admin workflows.

Member 2 Name: Jeremy Allan

- *Completed Section 3.1 Candidate Classes.* Spearheaded discussions regarding candidate classes. Developed initial candidate class list and facilitated refinement conversations.
- *Completed Section 3.3: CRC Cards* using examples provided in the weekly prereading & tutorial content. Defined Abstract relationships, Relationships and Collaborations for each candidate class.
- Designed the final UML diagram structure using draw.io. Compacted whitespaces in the diagram and simplified the workflows.
- Managed shared documents, including distribution, formatting and submission.
- Compiled evidence for Assignment 2 collaboration document.

Member 3 Name: Pulkit Pannu

- Completed Section 1 Executive Summary and Section 2 Problem Analysis and ensured that all the details were consistent based on the group discussions and the prior assignment requirements
- Assisted in discussions regarding the candidate classes for the system and did deeper analysis and reviews throughout.
- Updated the CRC cards with more detailed descriptions and ensured that their relationships were stated correctly.
- Reviewed and updated the Section 4 Quality of design to ensure that everything is consistent and elaborate more on the design patterns and heuristics.
- Reviewed and updated the bootstrap process of the system based on the design updates.

Member 4 Name: Christom Alappadan

- Completed Section 5 Wrote the bootstrapping process, outlining how the main system objects are created and connected.
- Completed Section 6 Documented key use scenarios, including checkout, guest account creation, sales reporting, order dispatch and catalogue browsing.

2. Description and evidence of systematic collaboration / teamwork on each significant work item (eg, discussions, mutual revisions and reviews, meeting minutes, etc.):

Evidence of initial task delegation:

general

 **Jeremy** 27/04/2026, 20:59
I think [@Christom](#) would be great at sections 4 & 5, based on his work for assignment 1
 2 

 **Justin (Stuff)** 27/04/2026, 21:00
okay so what are our allocations so far?
ill be happy to do 6 then

 **_pulkit_** 27/04/2026, 21:02

- Pulkit - section 1, 2 + UML for 3 and the dividing the section 3 load with Jeremy
- Jeremy - section 3
- Christom - 4, 5
- Justin - section 6

(edited)

does that work?

 **Justin (Stuff)** 27/04/2026, 21:03
you're doing the uml?

 **_pulkit_** 27/04/2026, 21:03
I don't mind at all if you wanna do that too
would be good is they're all similarly formatted
its just you have a decent load of 4 scenarios and their UMLs too
so don't wanna add too much to the list
upto you

Evidence of peer feedback/review:

general

Jeremy 03/05/2026, 16:28
@Christom I've already done a bit of your part so if you get through it easily, maybe check with Justin if there's anything extra you can help with?

Christom 03/05/2026, 18:55
Easy big man 🙌
👍 1

5 May 2026

Jeremy 05/05/2026, 22:37
@Justin (Stuff) a few notes on the draft UML: (edited)

Functional & Logic Adjustments

- **Guest Access:** Add an association line from Guest to ShoppingCart to show that guest users can manage a cart.
- **Admin Control:** Add association lines from Admin to ShippingManager and SalesReportGenerator, reflecting staff responsibilities identified in your requirements.
- **Fulfillment Flow:** Ensure the arrow points from ShippingManager to Courier to show the manager assigns tasks to the courier.
- **Extensibility:** Point the Catalogue association to the abstract Item class instead of directly to Book; this shows the system is designed to handle other products in the future.

Notation & Rubric Compliance

- **Stereotypes:** Explicitly label Address, SalesReport, and Invoice with a <<Data-Holder>> stereotype to satisfy the requirement for defining record-like abstractions.
- **Multiplicity:** Add cardinality labels (e.g., 1 to *) between Catalogue and Book, and Order and Book, to show the marker you understand the numerical relationships.
- **Aggregation vs. Composition:** Use a White Diamond (Aggregation) for the link between Catalogue and Book, as books exist independently of the catalogue.
- **Persistence:** Add dependency lines from User, Order, and Book to the DatabaseConnector to show these classes rely on the singleton for saving data.

👍 1

+ Database connector should also be accessed by many more classes

pulkit 05/05/2026, 23:01
Quick question about the UserFactory, I looked at the uni slides and they had the mention of Factory methods used on base classes rather than factory classes who's job is to just create stuff. So do we wanna follow that way? it's just this way we won't be able to show it in the diagram.... cuz its a method not a class

Pattern: Factory Method

Problem: How to define an interface to create objects, but defer the decision which specific class to instantiate?

Solution: Introduce a *Factory Method* in a base class (or interface) and delegate the decision which concrete class(es) to instantiate to subclasses (or implementing classes).

■ Factory Methods are often used in frameworks to properly encapsulate the instantiation of a derivations of a base abstraction.

Jeremy 05/05/2026, 23:01
That would simplify our diagram and reduce the objects, sounds good
So factory method in user class?

pulkit 05/05/2026, 23:02
yup
👍 1

Jeremy 05/05/2026, 23:02
Sounds good

Word document comments and feedback:

Version History

Comments

New

PP

PULKIT PANNU

...

👍

rename to catalogueManager?

JA

JEREMY ALLAN

...

👍

A catalogue is a thing though. Catalogue manager indicates that it is managing multiple catalogues.

@mention or reply

JC

JUSTIN CHEN

...

👍 1

yay or nay?

JA

JEREMY ALLAN

...

👍

We keep this, I removed the payment class. Payment method is now just a part of order.

@mention or reply

PP

PULKIT PANNU

...

👍

I think courier can be an attribute or an interface in the ShippingManager class and we can remove it?

View 1 more reply

JA

JEREMY ALLAN

...

👍

My plan was each courier would have a object (fedEx, AusPOST, etc). They then provide tracking to order. Shipping manager is designed to help staff pack the orders, not so much deliver them. It just assigns a courier to an order.

@mention or reply

PP

PULKIT PANNU

...

👍

Should we add an AccountManager class? This could handle user registration, auth, privileges and profiles etc.

View 1 more reply

Version History

Comments

New

JC

JUSTIN CHEN

I'm not entirely convinced an address could be a class, rather its just a string in the customer class

...

View 3 more replies

PP

PULKIT PANNU

lol yeahh ik what you mean, i was thinking the same but its more like a data struct than a class class so yeah it is needed

...

@mention or reply

PP

PULKIT PANNU

we should reword that to "a data-holder class" to be specific according to the assignment brief

...

JA

JEREMY ALLAN

Do we need more than what I've added here?

...

@mention or reply

PP

PULKIT PANNU

are all the necessary responsibilites of the system covered?
are the responsibilities described consistently and precisely?
Are the responsibilities "evenly" distributed or are "god" classes present?

...

@mention or reply

JC

JUSTIN CHEN

TODO: Simplify Scenarios, split if necessary. Bigger & easier to read.

...

JC

JUSTIN CHEN

Starting Points, End Points, Loops if necessary

...

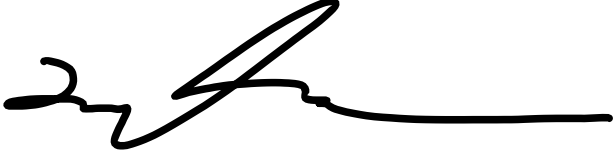
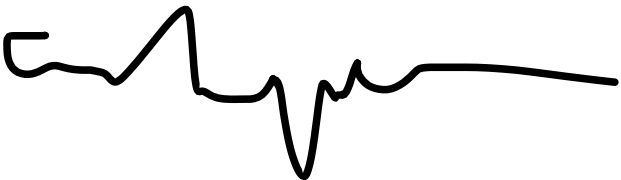
@mention or reply

Evidence of multiple revisions:

The entire word document history can be found here: https://liveswinburne.edu.au-my.sharepoint.com/:w/g/personal/104565600_student_swin_edu_au/IQBbMUpooaOyQ4xrlnlvt4gaAWBLX9Uif7E-d6XxzBGDAAU?e=PtM3gM

Version History		Comments
	Modified by: JEREMY ALLAN Open Version	12:40 pm
	Modified by: JEREMY ALLAN Open Version	12:39 pm
30 April 2026		
	Modified by: PULKIT PANNU Open Version	10:28 pm
	Modified by: PULKIT PANNU Open Version	10:08 pm
29 April 2026		
	Modified by: JEREMY ALLAN and JUSTIN CHEN Open Version	8:28 pm
	Modified by: JEREMY ALLAN and CHRISTOM ALAPPADAN Open Version	5:27 pm
	Modified by: JEREMY ALLAN Open Version	5:20 pm
	Modified by: JEREMY ALLAN and PULKIT PANNU Open Version	5:10 pm
	Modified by: JEREMY ALLAN and JUSTIN CHEN Open Version	4:56 pm
	Modified by: JUSTIN CHEN, JEREMY ALLAN and PULKIT PANNU Open Version	4:46 pm
	Modified by: JUSTIN CHEN Open Version	4:27 pm
	Modified by: JEREMY ALLAN and JUSTIN CHEN Open Version	3:47 pm
	Modified by: JEREMY ALLAN and JUSTIN CHEN Open Version	3:40 pm
	Modified by: JEREMY ALLAN and JUSTIN CHEN Open Version	3:30 pm
	Modified by: JEREMY ALLAN and JUSTIN CHEN Open Version	3:07 pm
27 April 2026		
	Modified by: JUSTIN CHEN Open Version	11:34 pm
	Modified by: JUSTIN CHEN Open Version	11:18 pm
	Modified by: JEREMY ALLAN Open Version	8:33 pm
	Modified by: JEREMY ALLAN Open Version	8:30 pm

3. Signed by all members:

ID Number	Name	Signature
<u>104565600</u>	<u>Jeremy Allan</u>	
<u>104093910</u>	<u>Pulkit Pannu</u>	
<u>101614596</u>	<u>Justin Chen</u>	
<u>103986022</u>	<u>Christom Alappadan</u>	

Software Requirements Specification Document

Favourite Books Online Bookstore System

SWE30003 - Assignment 1 - Group 3 - 16:30pm Wednesday

March 25, 2026

Contents

1	Introduction	4
2	Project Overview	5
2.1	Context	5
2.2	Project Type.....	5
2.3	Domain Vocabulary	5
2.4	Goals.....	6
2.5	Assumptions	6
2.6	Scope	6
3	Problem Domain	8
3.1	Pain Points	8
3.2	Domain Entities	8
3.3	Actors	8
3.4	Tasks	9
4	Functional Requirements and Task Descriptions	10
4.1	Task 1: Manage Customer Account	10
4.2	Task 2: Browse Book Catalogue	11
4.3	Task 3: Manage Shopping Cart	12
4.4	Task 4: Process Order and Payment	13
4.5	Task 5: Manage Packaging and Shipment.....	15
4.6	Task 6: Generate Sales Statistics	16
4.7	Task 7: Manage Book Catalogue	17
4.8	Task 8: View Order History	18
5	Workflow	20
5.1	Task 1 - Manage Customer Account	20
5.2	Task 2 - Browse Book Catalogue	21
5.3	Task 3 - Manage Shopping Cart	22
5.4	Task 4 - Process Order and Payment	23
5.5	Task 5 - Manage Packaging and Shipment.....	24
5.6	Task 6 - Generate Sales Statistics	25

5.7	Task 7 - Manage Book Catalogue.....	26
5.8	Task 8 – View Order History.....	27
6	Data Model	28
6.1	Domain Model.....	28
6.2	Entity Descriptions	28
6.3	Assumptions	29
6.4	Future Considerations	29
7	Quality Attributes of System	30
7.1	Security.....	30
7.2	Usability (User Experience)	30
7.3	Performance	31
7.4	Availability/Scalability	31
7.5	Extensibility (Modularity)	32
8	Other Requirements	33
8.1	Product level requirements.....	33
8.2	Design level requirements.....	33
9	Validation of Requirements	35
9.1	CRUD Check.....	35
10	Possible Solutions	37
10.1	Solution 1: Self-Hosted Web Application.....	37
10.2	Solution 2: Cloud-based Microservices Architecture	38
10.3	Solution 3: Commercial Off-The-Shelf (COTS) E-commerce Platform (Shopify).....	39

1 Introduction

This document presents a requirements specification for the design and implementation of an online component for 'Favourite Books', a bookstore on Glenferrie Road. The primary goal is to expand the business beyond its current physical operations in order to reach a broader online market.

This transition is motivated by the increasing demand for online storefronts and the competitive edge that they provide in a modern retail environment.

The aim of this document is to analyse the existing system and to define the requirements for the proposed online system by utilising the Tasks & Support approach. Through the analysis, key quality attributes will be identified, supporting the development of solutions to address the current system's limitations.

2 Project Overview

2.1 Context

Currently, 'Favourite Books' is a physical bookstore operating on Glenferrie Road. As with many traditional retail stores, its operations are limited to in-store interactions, restricting its ability to reach a wider audience and adapt to the growing trend of online retail.

To address the limitations, the proposed system introduces an online bookstore that enables customers to browse the book catalogue, create customer accounts, and manage shopping carts. In addition, the system will support core operational functions such as payment processing, packaging and shipment management, and maintaining the book catalog as new titles become available.

The system will also provide basic sales statistics over various time periods, allowing the business to analyse performance.

The objective of the development of an online bookstore is to improve accessibility while streamlining operations and expanding the store's reach beyond its physical location.

2.2 Project Type

Type: Information System + E-Commerce System

Users: External (Customers) + Internal (Staff)

The project can be classified as an e-commerce information system, utilising web elements. The project will be focusing on supporting both customer-facing interactions (browsing and purchasing) and internal business operations (order management & reporting).

2.3 Domain Vocabulary

- **Customer:** A user who interacts with the online bookstore to browse & purchase books
- **Catalogue:** The collection of books available for browsing and purchase
- **Shopping Cart:** A temporary collection of selected books prior to checkout
- **Order:** A confirmed purchase containing selected books
- **Shipment:** The process of packaging and delivering books to customers
- **Invoice:** Financial documentation generated containing payment & order information.
- **Sales Report:** Aggregated sales data over a period of time.
- **Payment:** A financial transaction made by a customer to complete an order.
- **Payment Processor:** An external financial service responsible for authorising, validating and processing customer payments during checkout.

2.4 Goals

The goals of the proposed online bookstore systems are as follows:

- Expand the business reach from a local area to an Australian online market.
- Provide a platform for customers to browse and purchase books.
- Provide the ability to create and manage user accounts for an individual and streamlined shopping experience.
- Support order processing, which includes shopping cart management and checkout.
- Process orders with payment processing, management of packaging and shipping and invoice printing.
- Maintain, manage and update an up-to-date catalogue of books
- Provide basic statistics about books sold over certain periods for analysis.

2.5 Assumptions

- Payment processing will be handled by a third-party payment provider.
- Delivery and Shipment will be handled by external courier services.
- Customers will have access to internet-enabled devices.
- Customers will have basic digital literacy.
- Inventory data is assumed to be accurate and managed by store staff.
- The system will not handle physical inventory discrepancies in real-time (handled manually by staff).
- External services (payment and courier APIs) are assumed to be reliable but may occasionally fail.

2.6 Scope

In Scope:

- Browsing and searching a book catalogue.
- Creating and managing customer accounts.
- Managing Shopping Carts and placing orders.
- Processing online payments.
- Handling order fulfilment, including packaging and shipment coordination.
- Maintaining and updating the book catalogue.
- Generating basic sales statistics and reports.

Out of Scope:

- Loyalty Programs.
- Rent-before-Buying.
- Returns and return shipping.
- Advanced recommendation/marketing algorithms.
- Control or assistance over physical operations, including delivery logistics.

These exclusions ensure that the system remains focused on core transactional functionality while allowing future extensibility

3 Problem Domain

3.1 Pain Points

- **Limited customer reach:** Currently, Favourite Books operates solely as a physical store, meaning its customer reach is limited to those in the surrounding suburbs with word-of-mouth exposure.
- **Missed revenue opportunities:** By not having an online store, the business is missing out on the growing trend of online shopping and failing to reach customers Australia-wide.
- **Lack of automated sales analytics:** The owners currently lack an automated system to generate basic statistics about books sold over specific time periods (e.g., daily, weekly, monthly, yearly) to oversee business operations effectively.

3.2 Domain Entities

- Customer
- Book / Catalogue
- Shopping Cart
- Order
- Payment
- Invoice / Receipt
- Shipment / Package
- Sales Report

3.3 Actors

- **Customer:** The primary user who browses the catalogue, manages their shopping cart, and places orders.
- **Store Owner / Manager:** The business operator who views basic statistics about books sold and manages the types of books available for sale.
- **Fulfillment Staff:** The employees responsible for managing the packaging and shipment of the books.
- **Payment Processor (External Actor):** The external financial gateway required for handling and verifying payments.

- **Courier / Delivery Service (External Actor):** The external service responsible for physically delivering the packaged books to the customer's home.

3.4 Tasks

ID	Task	Description
1	Create Customer Account	Users can create and maintain individual accounts to store personal data.
2	Browse Book Catalogue	Users can browse and search the available catalogue of books.
3	Manage Shopping Cart	Users can add, update or remove books from a cart for purchase.
4	Process Order and Payment	In the checkout process, processing payments and creating customer invoices/receipts is automatically handled.
5	Manage Packaging and Shipment	The system provides staff the details required for the physical preparation, packaging, and dispatch of customer orders.
6	Generate Sales Report	Generating basic statistical reports about books sold over various periods (a day, a week, a month, a year, or year to date).
7	Manage Book Catalogue	Enabling site admin to update the types of books available for purchase as new books arrive.
8	View Order History	Interface allows customers to view the delivery status and location of current orders plus view a record of past purchases online.

4 Functional Requirements and Task Descriptions

4.1 Task 1: Manage Customer Account

Task: Manage Customer Account	
Purpose:	Allow customers to create and manage their personal profiles, saving details for future orders. This will streamline the purchase process for returning customers.
Trigger/Precondition	A user decides they want to create an account to save their details or track an order. To do this, they click “Sign Up” or “My Account”.
Frequency	Moderate (Daily).
Critical:	Forgotten passwords preventing access and unauthorized account takeovers.
Work Area:	Customer’s personal device. PC or mobile device.
Subtasks:	Example Solution:
1. Register a new account. <i>Problem: Email is already registered.</i>	The system provides a registration form capturing name, email, and password. System checks database; if email exists, prompts the user to log in or reset password instead.
2. Update shipping/billing address.	The website provides a profile dashboard where users can add, edit, or delete saved addresses and other personal details.
Variants	
1a. Guest checkout and conversion	Users can choose to checkout as a guest. After a guest purchase, the system offers a 1-click option to create an account using the provided order details.

4.2 Task 2: Browse Book Catalogue

Task: Browse Book Catalogue	
Purpose:	Enable customers to explore, search, and view the available catalogue of books.
Trigger/Precondition	A customer navigates to the online bookstore homepage.
Frequency	Very High (Hundreds of times per day).
Critical:	Search functionality crashes or returns irrelevant results.
Work Area:	Customer's personal device.
Subtasks:	Example Solution:
1. Search for a specific book. <i>Problem: Customer misspells the title or author.</i>	System provides a search bar that dynamically presents book titles that match the inputted text. System uses a fuzzy-search algorithm to suggest "Did you mean...?" results.
2. Browse book by genres.	If a title is not inputted in the search bar, recommended/popular books are displayed. Users can sort these books by genre using a dropdown filter.
3. View book details.	The interface retrieves and displays the book's title, cover image, synopsis, price, and current stock availability.
Variants	
3a. Book is out of stock	The website greys out the "Add to Cart" button and offers a "Notify me when available" email alert option instead.
3b. Add book to Wishlist	A love heart shaped icon is displayed along with the text "Add to Wishlist". Once selected, books will be stored in a separate collection that users can refer to later.

4.3 Task 3: Manage Shopping Cart

Task: Manage Shopping Cart	
Purpose:	Allows customers to collect, review, and modify the books they intend to purchase.
Trigger/Precondition	Customer clicks “Add to Cart” on a book’s detail page.
Frequency	High.
Critical:	Cart data is lost if the customer accidentally refreshes the page or closes the browser.
Work Area:	Customer’s personal device.
Subtasks:	Example Solution:
1. Add book to cart.	The interface visually confirms the addition (eg. a pop-up or cart icon update) and temporarily reserves the inventory.
2. Update quantities or remove items. <i>Problem: Customer requests more copies than are currently in stock.</i>	System provides a cart summary page with “+” / “-” toggles and a “Remove” button, dynamically updating the total price displayed. System limits the maximum selectable quantity to the current inventory level and displays a warning.
Variants	
1a. Saved cart	When logged in, the system utilizes browser cookies or account data to save the cart state so the user can resume shopping later.
1b. Add recommendations to cart	After a book has been added to the cart, a list of similar suggestions (determined by a shared genre, ratings, etc) is offered to the user. A prompt saying, “We think you’d like these too!” is displayed.

4.4 Task 4: Process Order and Payment

Task: Process Order and Payment	
Purpose:	Finalize the purchase, securely process payments, and generate invoices/receipts.
Trigger/Precondition	Customer reviews their cart and clicks "Checkout".
Frequency	Moderate to High (depending on sales volume).
Critical:	External payment gateway goes down; fraudulent transaction attempts.
Work Area:	Customer's personal device.
Subtasks:	Example Solution:
1. Enter shipping details.	The UI displays a form for delivery info (or auto-fills for logged-in users).
2. Process payment. <i>Problem: Credit card is declined.</i>	The system securely transmits payment data to a 3rd party financial processor. The process halts the checkout, does not create an order, and prompts the user to try a different card.
3. Save transaction and generate Order Number.	Following successful payment, the order details are uploaded to the system database, and a unique Order Number is created. A success prompt displaying the Order Number is visible on the interface.
4. Generate financial records.	Using the Order Number, the system creates a tax invoice and receipt, emails them to the customer, and logs the order.
Variants	
1a. Billing data matches Shipping data	System provides a checkbox to duplicate shipping info into the billing fields to save user time.
1b. Guest checkout personal details	If a user selects guest checkout, they will be prompted to provide some basic personal details (eg. Name, Email Address) before entering shipping details.

	These will only be used for sending proof of purchase (receipts) and identifying the customer in necessary communications.
--	--

4.5 Task 5: Manage Packaging and Shipment

Task: Manage Packaging and Shipment	
Purpose:	Facilitate the physical preparation, packaging, and dispatch of customer orders.
Trigger/Precondition	An order is successfully paid and recorded in the system.
Frequency	Moderate (Daily batch processing by staff).
Critical:	Shipping the wrong book to the wrong customer.
Work Area:	Physical Bookstore / Backroom (Staff PC).
Subtasks:	Example Solution:
1. View pending orders.	System provides a staff dashboard listing all orders marked "Paid - Awaiting Shipment".
2. Prepare packaging materials. <i>Problem: Address is invalid or incomplete.</i>	The system generates a printable packing slip and shipping label. Staff member is notified of the book's location within the store/backroom. Orders are flagged with unverified addresses prior to label generation, prompting staff to contact the customer.
3. Dispatch order.	Staff input the courier tracking number in the field associated with the pending order. The system updates the order status to "Shipped" and emails the customer. Further tracking notifications are handling by the courier.
Variants	
3a. Partial shipment	If a book is physically missing despite system inventory, staff can mark the order as "Partially Shipped" and automatically trigger a refund or backorder for the missing item.

4.6 Task 6: Generate Sales Statistics

Task: Generate Sales Statistics	
Purpose:	Provide business owners with overview statistics about books sold over various periods.
Trigger/Precondition	Store owner logs into the administrative dashboard and navigates to "Reports".
Frequency	Low (Daily, weekly, or monthly).
Critical:	System lag when querying large amounts of historical sales data.
Work Area:	Store Owner's PC (Back office).
Subtasks:	Example Solution:
1. Select reporting parameters.	System provides dropdowns and date-pickers to select periods (a day, week, month, year, or YTD).
2. Generate and view report. <i>Problem: Required data set is too large to render quickly.</i>	System queries the database and generates visual charts and data tables of sales performance. System processes the report asynchronously and emails a download link to the manager when ready.
Variants	
2a. Exporting data	System allows the owner to export the generated statistics as a PDF or CSV file for accounting purposes.
2b. Auto-Generate reports	System automatically generates sales reports based on a specified timeframe (monthly, etc) and emails the report directly to the owner.

4.7 Task 7: Manage Book Catalogue

Task: Manage Book Catalogue	
Purpose:	Allow management to change and update the types of books available for sale as new books arrive.
Trigger/Precondition	New inventory arrives at the physical store, or pricing needs adjustment.
Frequency	Low to Moderate (Weekly).
Critical:	Accidental deletion of an active, high-selling book record.
Work Area:	Store Owner's PC.
Subtasks:	Example Solution:
1. Add new books to the system.	System provides a data entry form to input book details (Title, ISBN, Price, Stock level, Cover Image, etc).
2. Update existing books. <i>Problem: A book goes out of print permanently.</i>	The interface provides a searchable list of current inventory with inline editing capabilities. Instead of deleting the record (which breaks historical sales reports), the system allows the owner to mark the book status as "Archived/Discontinued". The book will now appear greyed-out on the Browse interface.
Variants	
1a. Bulk Import	System provides a feature to upload a CSV file from a publisher to automatically populate multiple new books at once.

4.8 Task 8: View Order History

Task: View Order History	
Purpose:	Allow customers to browse their purchase history, review past order details (like invoices), and track the delivery status of active orders.
Trigger/Precondition	Customer navigates to their account dashboard, or clicks an order link in their email.
Frequency	High (Customers often check multiple times per order).
Critical:	Database timeout when retrieving extensive user purchase history or external courier API is unresponsive.
Work Area:	Customer's personal device.
Subtasks:	Example Solution:
1. Access order history list.	System provides a "My Orders" dashboard displaying a chronological list of past and current orders, including quick-status badges (e.g., "Delivered", "Processing", "In Transit").
2. View specific order details.	System retrieves full order information upon selection, including purchased books, total price paid, and a button to view/download the original tax invoice.
3. Track active delivery status. <i>Problem: Courier API is unresponsive.</i>	For orders currently in transit, the system queries the Courier API for real-time location updates. If the API fails, the system gracefully degrades, showing the last known internal status and providing a direct link to the courier's website to check manually.
Variants	

1a. Guest user lookup	Because guests do not have an account dashboard, the system provides a public "Track/View Order" portal requiring the Order Number and Email Address to view a specific past order's details and status.
-----------------------	--

5 Workflow

5.1 Task 1 - Manage Customer Account



Figure 1: Manage Customer Account Workflow Diagram

5.2 Task 2 - Browse Book Catalogue

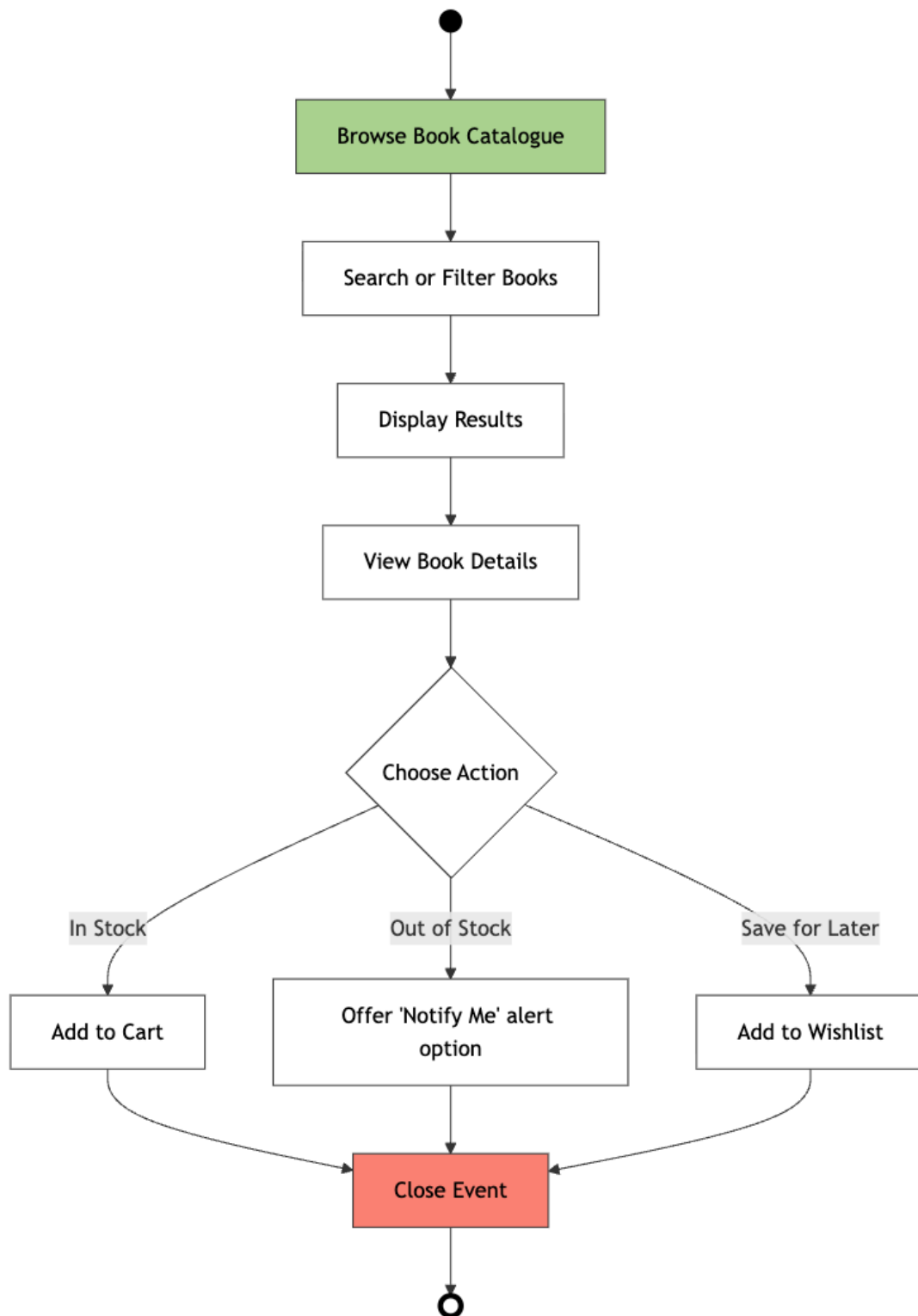


Figure 2: Browse Book Catalogue Workflow Diagram

5.3 Task 3 - Manage Shopping Cart

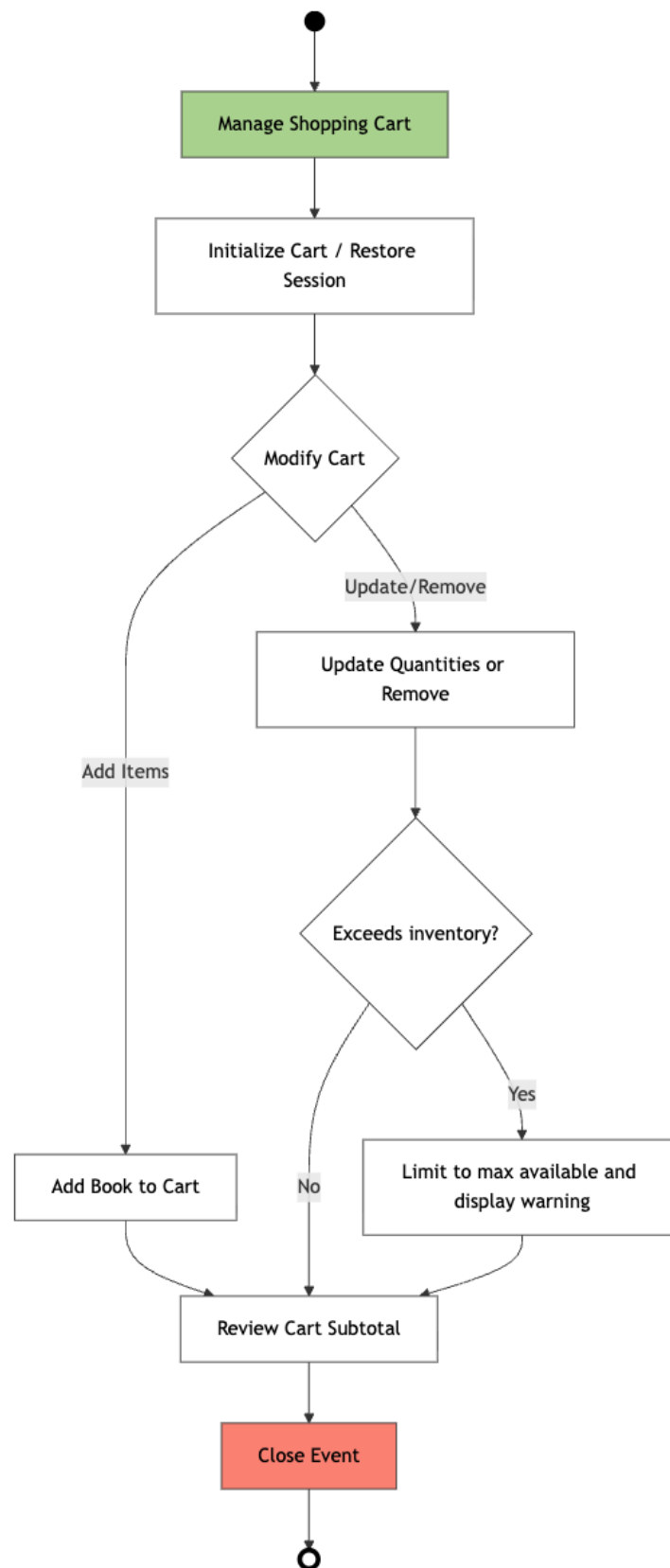


Figure 3: Manage Shopping Cart Workflow Diagram

5.4 Task 4 - Process Order and Payment

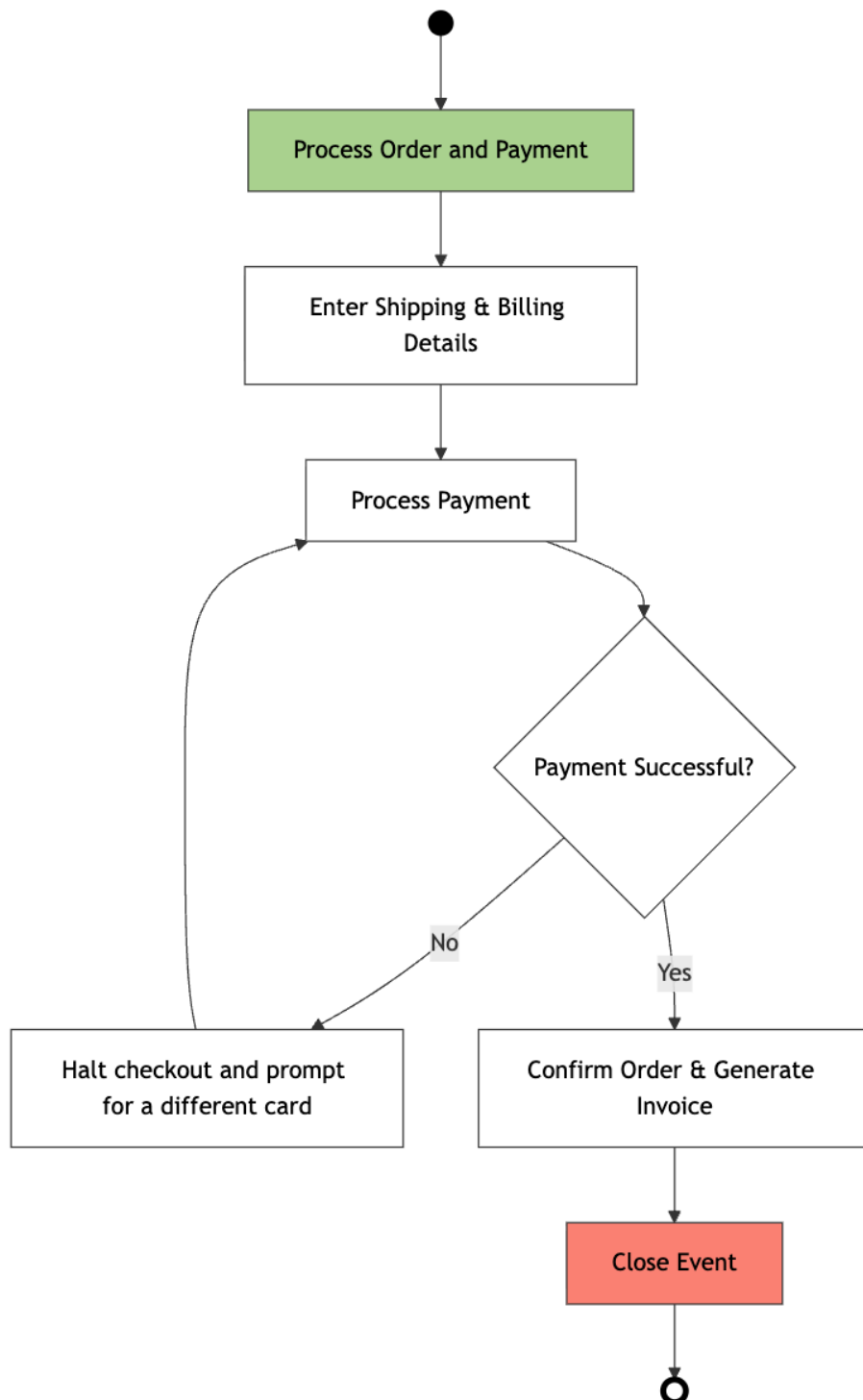


Figure 4: Process Order and Payment Workflow Diagram

5.5 Task 5 - Manage Packaging and Shipment

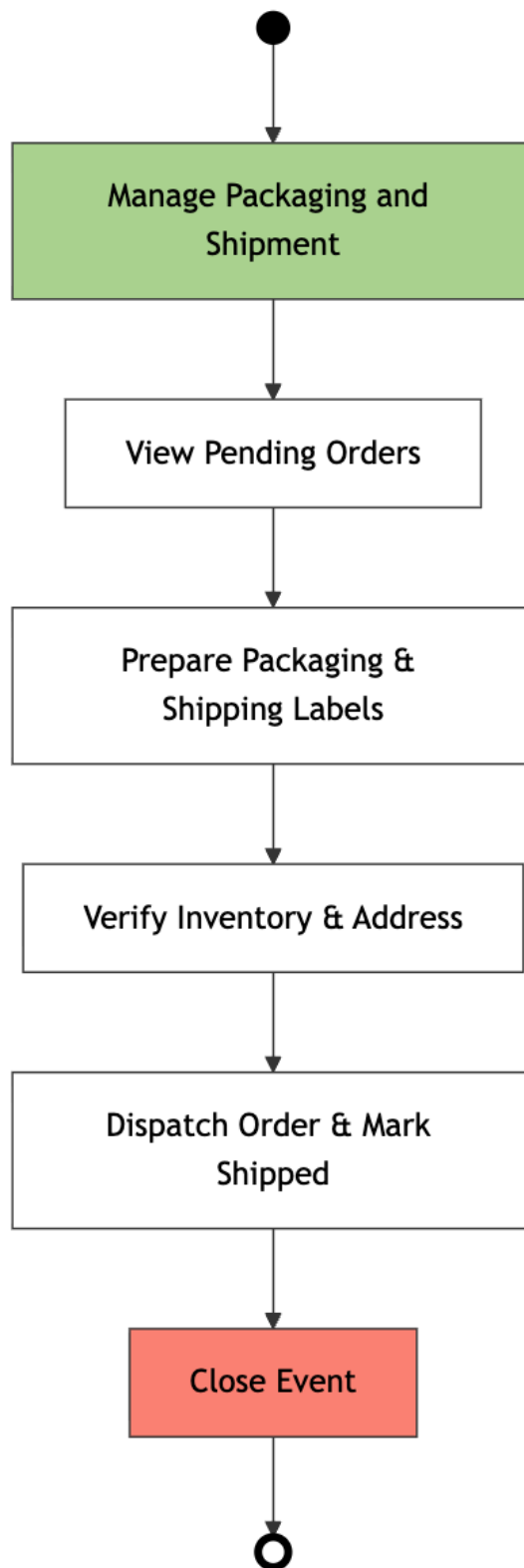


Figure 5: Manage Packaging and Shipment Workflow Diagram

5.6 Task 6 - Generate Sales Statistics

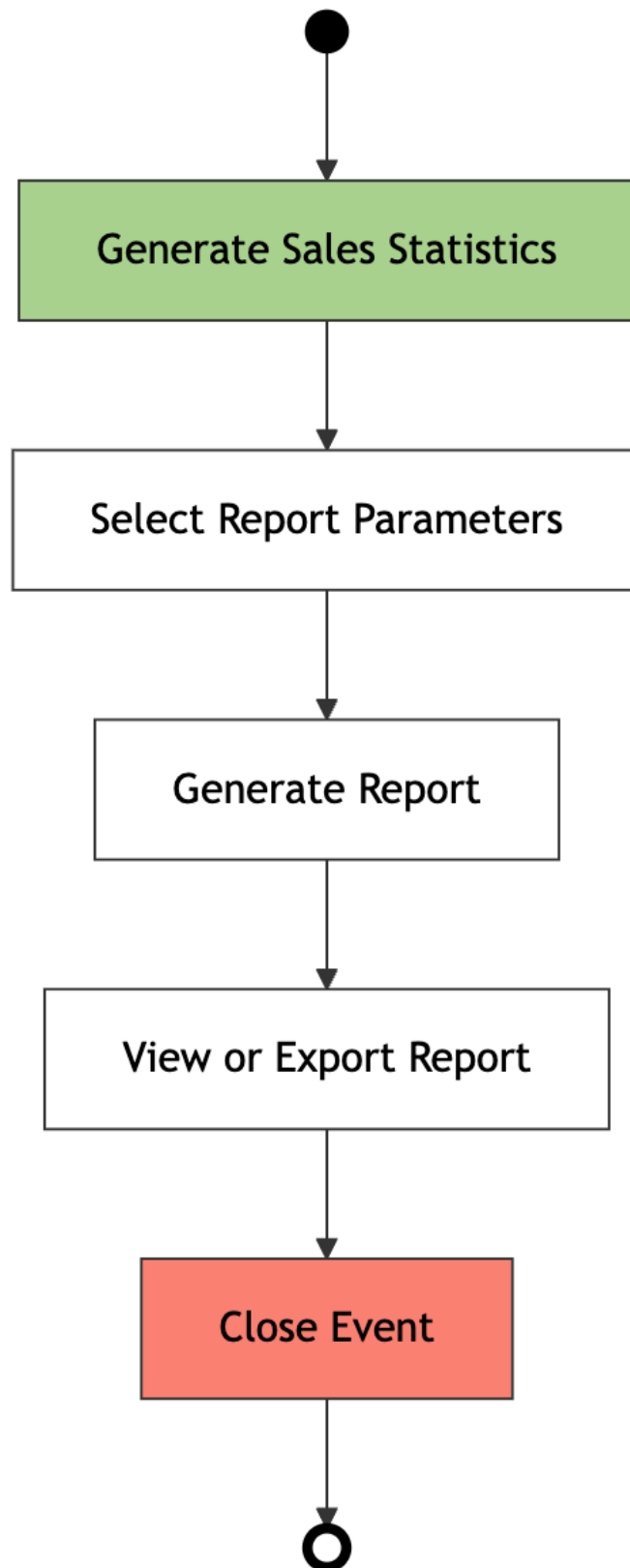


Figure 6: Generate Sales Statistics Workflow Diagram

5.7 Task 7 - Manage Book Catalogue

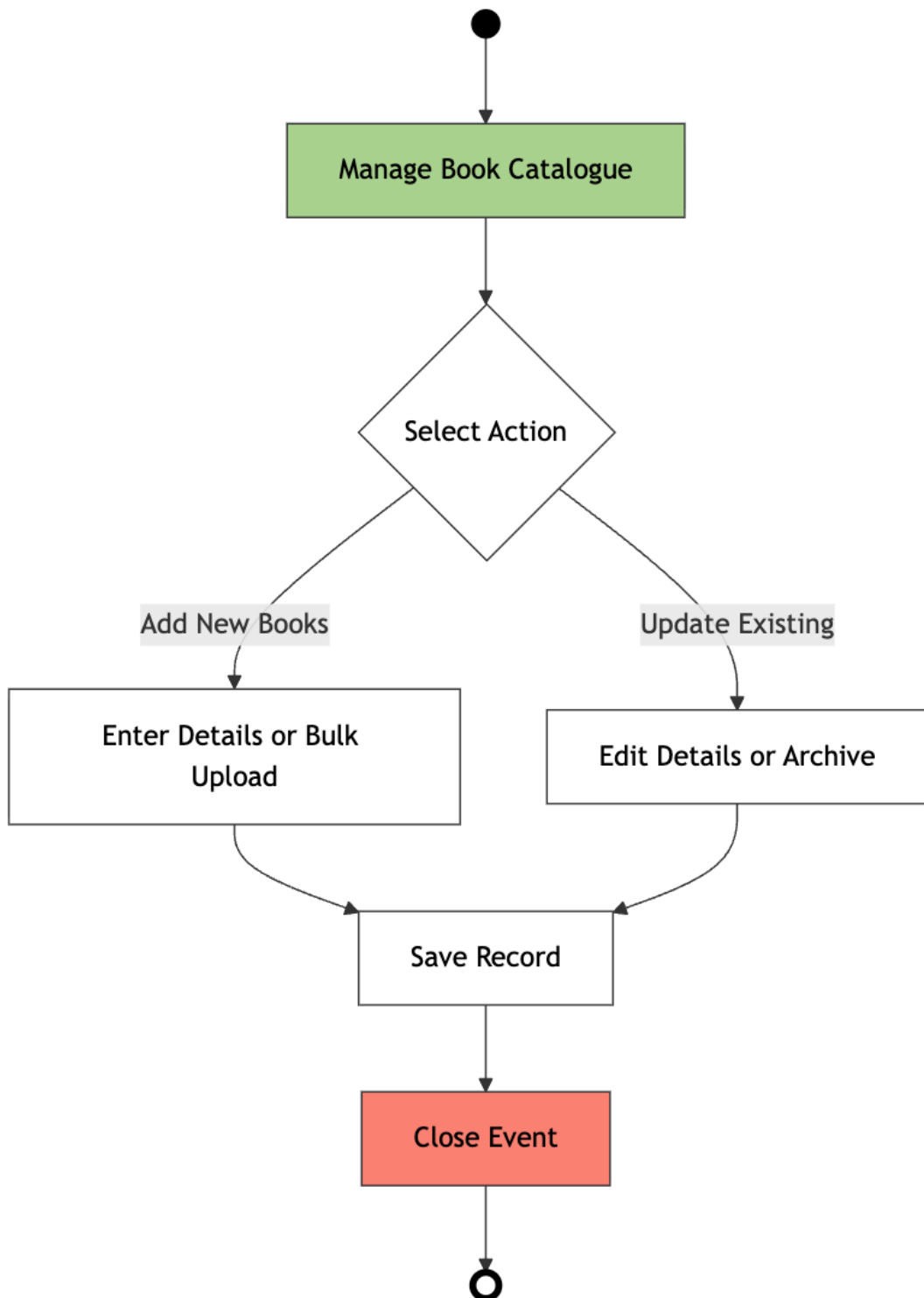


Figure 7: Manage Book Catalogue Workflow Diagram

5.8 Task 8 – View Order History

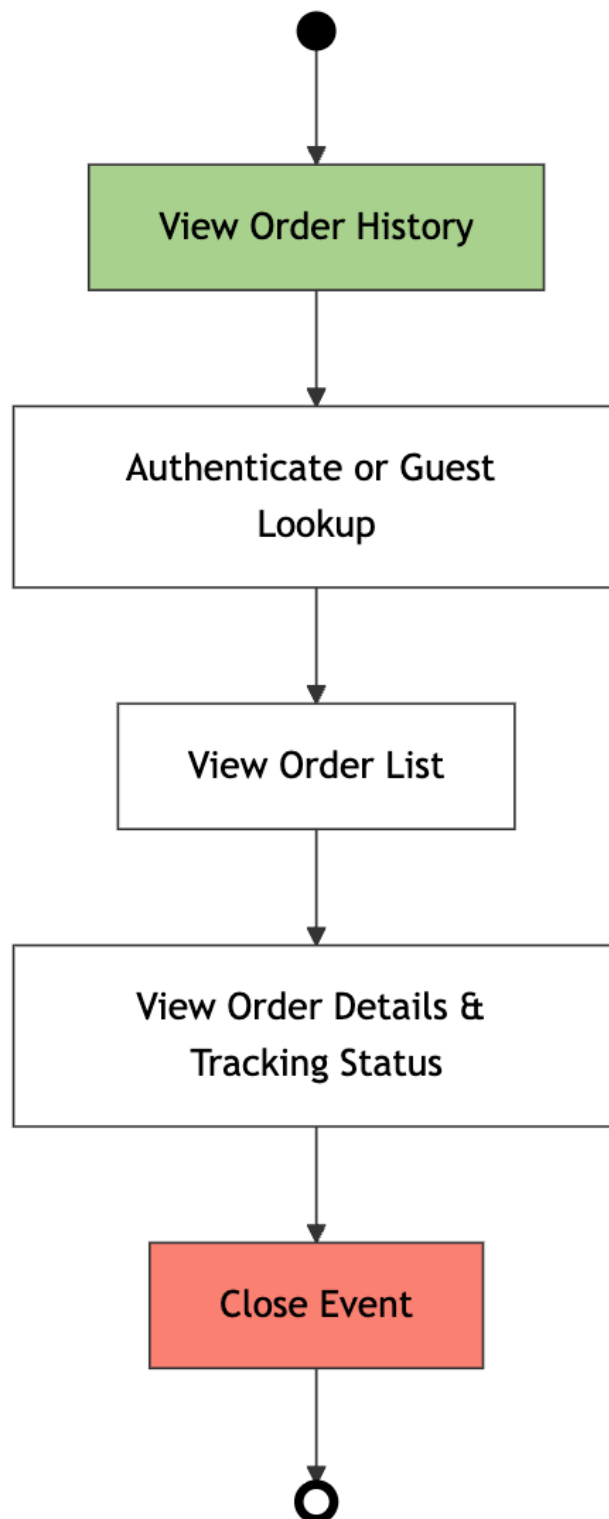


Figure 8: View Order History Workflow Diagram

6 Data Model

6.1 Domain Model

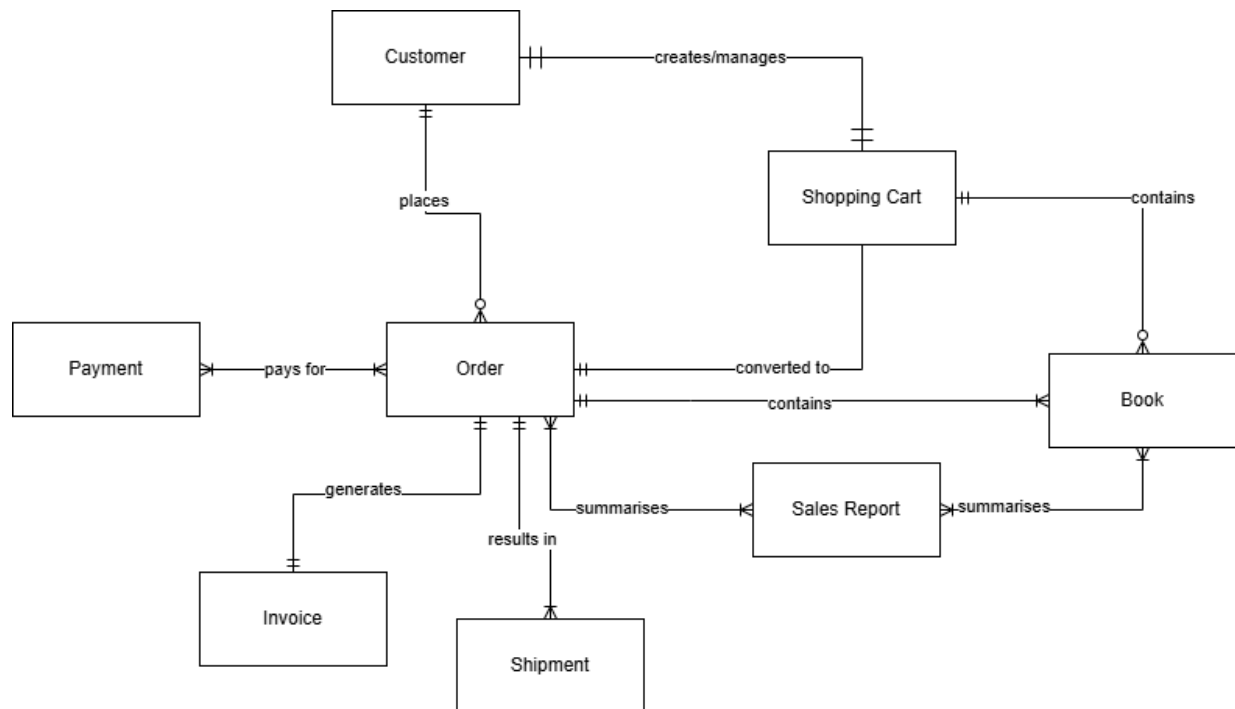


Figure 9: Domain Model Diagram

The domain model illustrates the key entities within the online bookstore system and the relationships between them. It focuses on conceptual interactions rather than implementation details. The model highlights how customer actions drive the creation of orders, which in turn trigger payments, invoicing and shipment processes, while also contributing to sales reporting.

6.2 Entity Descriptions

- **Customer:** A user of the online bookstore who can browse books, manage a shopping cart, places orders, makes payments.
- **Shopping Cart:** A temporary collection of books selected by a customer before checkout. It allows the customer to review intended purchases, update quantities, remove books prior to making an order, which will copy the state of the shopping cart.
- **Order:** A representation of a customer's confirmed purchase after checkout from the shopping cart. It is used as the basis for payment processing, invoicing and shipping.
- **Payment:** The financial transaction made for an order. It is processed through an external payment service and must be successfully verified before the order is finalised.

- **Invoice:** A financial record generated from a completed order. Its purpose is to provide proof of purchase for the customer & supports the business's financial record-keeping.
- **Shipment:** The representation of the physical fulfilment and dispatch of the customer's order. It is created after payment is confirmed.
- **Sales Report:** A summary of sales information gathered over a selected time period, such as weekly, monthly, or year to date. It helps the store owner or manager review business performance and trends in book sales.
- **Book:** An item offered for sale through the bookstore. Books appear in the catalogue and can be added to shopping carts, included in orders, and contribute to sales reporting.

6.3 Assumptions

- A customer will always have a shopping cart active.
- At checkout, Shopping Cart's contents are transferred into a new order (which includes quantities, items[books])
- A shopping cart may or may not be converted into an order.
- Each shipment is linked to one order only.
- The system will support multiple shipments per order

6.4 Future Considerations

The next stage of Requirements specification involves object-oriented design. These are just some things to keep in mind as the project grows.

- Extensibility:
 - Shall books be a subclass of an Item class? In case the business decides to expand past primarily selling physical books.
 - Support for multiple item types (e.g. eBooks, audiobooks) beyond physical books
 - Introduction of loyalty programs or discounts (out-of-scope)
 - Integration with recommendation systems or personalised suggestions.
 - Support for partial order cancellations or returns (currently out-of-scope)

7 Quality Attributes of System

7.1 Security

The online bookstore system should have robust abilities to protect data and user information from unauthorized access and ensure that the service remains available to legitimate users. Now since this new online system will be responsible for handling payments and creating/storing customer accounts, security becomes vital and a top priority for the system to ensure that it maintains customer trust.

A major benefit of prioritizing security is the prevention of data breaches. For instance, by using secure payment gateways, the system ensures that a customer's financial details, personally identifiable information (PII), are never stored in a vulnerable format, which protects both the customer and the bookstore's reputation.

To meet this quality attribute, the system must:

- Implement robust encryption and secure authentication protocols.
- Ensure that it is PCI DSS (Payment Card Industry Data Security Standard) compliant and that the payment gateways are secure.
- Ensure that any customer data stored is encrypted and protected with SSL.

7.2 Usability (User Experience)

It is imperative that the Favourite Books online store is easy to use and efficient for the customers, allowing them to smoothly browse the catalogue, manage their shopping cart and receive payment confirmations and receipts. Since Favourite Books is moving from a local physical store to a broader online presence, the online interface needs to be intuitive enough for people who are used to physical shopping to transition to digital shopping without frustration. Unsatisfying or bad UI/UX can lead to customer dropouts.

An example of this is if the UI is designed with clear navigation and search filters, it will reduce the so called "user friction", making it more likely that a local regular will successfully complete a purchase online rather than giving up and going to a larger competitor such as amazon.

To meet this quality attribute, the system must:

- Have mobile responsiveness as majority of people use their phones to search and find books and bookshops online.
- Ensure that the site is usable for everyone, regardless of their impairments, by following Web Content Accessibility Guidelines (WCAG).

- Have logical categories allowing for intuitive and quick searches and enable the users to quickly sort the results based on their requirements, such as by price, date, author, etc.

7.3 Performance

Performance refers to the responsiveness and speed of the system when a user is interacting with it, such as how quickly the book catalogue loads or how fast a search query returns results. For the Favourite Books online store, maintaining high performance is very important as the business is expanding to a national online presence. If the website is slow or laggy while customers are browsing books, it creates a negative impression and a bad user experience, potentially leading to high bounce rates, especially because many online shoppers expect instant results.

The main benefit of optimizing performance is the improvement of the overall customer experience and conversion rates. For example, if the system can process and provide results almost instantly, even if there are thousands of books and titles in the database, ensures a good customer experience even as the inventory grows or as more users join the platform.

To meet this quality attribute, the system must:

- Ensure that page load times for the book catalogue and search results remain under 2 seconds by properly indexing the databases.
- Optimize all book cover images and media files to ensure they do not slow down the site's responsiveness.
- Implement efficient database queries so that generating invoices and receipts does not lag during peak hours.

7.4 Availability/Scalability

Availability and scalability are two interlinked attributes that focus on the system's uptime and its capacity to handle growth. Availability ensures that the online bookstore is always accessible and available, which is a major shift from their physical store which is only open during set hours. Scalability, on the other hand, is the system's ability to handle increasing number of users as the online store starts to grow and reach customers all over Australia.

These two go hand in hand because if the system cannot scale to meet high traffic, such as during a big sale or a viral social media post, then the availability will suffer making the site may crash or become unresponsive.

To meet this quality attribute, the system must:

- Be hosted on a cloud infrastructure that supports auto-scaling to handle fluctuating traffic from across the country.

- Maintain an uptime of at least 99.9% of the time.
- Implement load balancing and redundancy to distribute traffic during peak hours evenly across multiple servers and prevent any single point of failure.

7.5 Extensibility (Modularity)

Extensibility or modularity is basically how easily a system can grow or be changed without having to make any major changes or start over from scratch. In the case of Favourite Books, the owners have already mentioned features like loyalty programs, discounts and "rent-before-buying" as things they might want to add later, even though they aren't part of the initial project. By designing the system to be modular from the start, we're making sure that the basic version is flexible enough to incorporate these new functionalities with ease when the business is ready.

The main advantage of this approach is that it makes the bookstore ready for the future and expansion and saves a lot of money on technical work down the road, enabling them to be competitive and adapt to new trends quickly if needed.

To meet this quality attribute, the system must:

- Have separate modules for each functionality so they can be updated or changed independently.
- Ensure the database is flexible enough to handle "new books" and different "types of books" as the inventory evolves.
- Use clear internal APIs so that future ideas, like loyalty programs, can be integrated with minimal hassle.

8 Other Requirements

8.1 Product level requirements

In addition to supporting the defined user tasks, the system shall satisfy the following product-level requirements:

- The system shall securely store and manage all user and transactional data, including customer details, orders, and payment confirmations.
- The system shall maintain data consistency across all components, ensuring that updates to inventory, orders, and customer information are reflected system-wide in real time.
- The system shall integrate reliably with external services, including payment gateways for transaction processing and courier services for shipment tracking.
- The system shall support concurrent access by multiple users, including customers and staff, without causing data conflicts or performance issues.
- The system shall maintain a complete audit trail of transactions, ensuring that all orders, payments, and updates are recorded and traceable.
- The system shall include error handling and recovery mechanisms, such as retrying failed transactions or notifying users when an action cannot be completed.
- The system shall support data backup and recovery to prevent loss of critical information.

8.2 Design level requirements

The system design shall follow both internal standards and widely accepted best practices for modern web applications:

- The system shall comply with Swinsoft UI/UX design guidelines, as outlined in the assignment specification.
- The system shall be implemented as a web-based application accessible through modern browsers on both desktop and mobile devices.
- The system shall follow a modular architecture that separates the user interface, business logic, and data storage, improving maintainability and allowing future extensions.
- The system shall support responsive design, ensuring the interface adapts effectively to different screen sizes and devices.
- The system shall provide clear navigation and consistent interface elements to support usability across all tasks.
- The system shall use secure communication protocols (e.g., HTTPS) for all data transmission.
- The system shall ensure compatibility with major web browsers (e.g., Chrome, Safari, Edge).

- The system shall allow for future extensibility, such as the addition of loyalty programs or recommendation systems (currently out of scope).

9 Validation of Requirements

A CRUD (Create, Read, Update, Delete) validation was conducted to ensure that all domain entities are appropriately used across the system's tasks. This helps confirm that each entity plays a meaningful role and that no unnecessary or unused components exist.

The table below demonstrates how each task interacts with the system's entities. It shows whether data is created, read, updated, or deleted during each process.

This validation confirms that:

- All entities are utilised across at least one task
- All core tasks interact with relevant system data
- There are no gaps or inconsistencies between the defined tasks and the data model

9.1 CRUD Check

Task	Customer	Book / Catalogue	Shopping Cart	Order	Payment	Invoice / Receipt	Shipment / Package	Sales Report
Manage Customer Account	C, R, U							
Browse Book Catalogue		R						
Manage Shopping Cart	R	R	C, R, U, D					
Process Order and Payment	R	R	R	C	C	C		
Manage Packaging and Shipment	R	R		R, U		R	C, U	
Generate Sales Statistics		R		R	R	R	R	C, R
Manage Book Catalogue		C, R, U						

Track Order Status	R	R		R	R	R	R	
-----------------------------------	---	---	--	---	---	---	---	--

10 Possible Solutions

10.1 Solution 1: Self-Hosted Web Application

This solution proposes a centrally managed, self-hosted web application where all system components are deployed on a single server or a small number of managed servers. The system would provide a unified platform for both customers and staff to interact with the online bookstore.

This architecture supports the core user tasks identified in this specification, including browsing the book catalogue, managing customer accounts, processing orders and payments, and supporting packaging and shipment workflows. All functionality is handled within a single integrated system, allowing consistent access to shared data such as customer details, inventory, and order records.

As the store is transitioning from a purely physical operation to an online presence, a self-hosted system provides a relatively straightforward and cost-effective approach to implementation, therefore, this solution aligns well with the current scale of the business. It enables the business to quickly establish an online storefront without requiring complex infrastructure or advanced technical expertise.

Simplicity is a key strength of this approach, this is demonstrated through the tightly integrated components allowing system design, deployment, and maintenance to be easier to manage. It also allows for strong control over data and system behaviour, which is beneficial for ensuring consistency across tasks such as order processing and inventory updates.

However, this solution introduces limitations in terms of scalability and availability. As the number of users increases, the system may experience performance bottlenecks, particularly during high-demand periods. Additionally, a failure in the central system could impact all functionality, reducing system reliability.

Overall, the self-hosted web application is most suitable as an initial solution for Favourite Books, providing a practical and manageable way to establish an online presence while supporting all required business operations.

Solution 1 Pros and Cons Table:

Pro/Con	Details
Pros – Simplicity	Easier to design, develop, and manage as everything is contained within a single system

Pros – Development Speed	Faster to implement, making it suitable for an initial rollout
Pros – Cost	Lower initial setup and operational costs compared to cloud-based solutions
Pros – Data Consistency	Centralised data storage makes it easier to maintain consistency across tasks
Cons – Scalability	Difficult to scale efficiently as user demand increases
Cons – Performance	Performance may degrade under high traffic or peak usage
Cons – Reliability	Single point of failure can affect the entire system
Cons – Flexibility	Harder to extend or add new features in the future

10.2 Solution 2: Cloud-based Microservices Architecture

This solution proposes a cloud-based microservices architecture, where the system is divided into independent services, each responsible for a specific business function. This solution is characterised by separate services being used for customer account management, book catalogue management, order processing, payment handling, shipment coordination, and sales reporting.

This approach supports the identified user tasks by allowing each functional area of the system to operate independently while communicating through defined interfaces. This is demonstrated when the order processing service can interact with the payment service and shipment service without being tightly coupled, thereby enabling more flexible and modular system behaviour.

This solution is particularly suitable for Favourite books because it allows for long-term growth and scalability. If the business expands nationwide, the system can handle increased traffic more effectively by scaling individual services rather than the entire system. It also allows new features, such as promotional systems or advanced analytics, to be added without significantly impacting existing functionality.

A major strength of this architecture is its scalability and resilience. Failures in one service are less likely to impact the entire system, improving overall availability. Additionally, cloud-based deployment allows the system to dynamically allocate resources based on demand, ensuring consistent performance during peak usage.

However, this solution introduces increased complexity in system design, development, and maintenance. Managing multiple services, handling communication between them, and ensuring

data consistency across services requires more advanced technical expertise. It may also lead to higher operational costs compared to a simpler self-hosted system.

Ultimately, the cloud-based microservices architecture is well suited for a scalable, future-oriented solution, but may be more appropriate as a later-stage implementation once the business grows beyond the capabilities of a simpler system.

Solution 2 Pros and Cons Table:

Pro/Con	Details
Pros – Scalability	Individual services can scale independently based on demand
Pros – Performance	Better handling of high traffic and large workloads
Pros – Reliability	Failures are isolated to specific services, reducing system-wide impact
Pros – Flexibility	Easier to add, modify, or extend features without affecting the entire system
Cons – Complexity	More complex to design, develop, and manage due to multiple services
Cons – Development Time	Longer initial development time compared to a single system
Cons – Cost	Higher operational costs due to cloud infrastructure and service management
Cons – Maintenance	Requires ongoing management of multiple services and their interactions

10.3 Solution 3: Commercial Off-The-Shelf (COTS) E-commerce Platform

This solution proposes adopting a Commercial Off-The-Shelf (COTS) e-commerce platform, such as Shopify, to serve as a fully managed foundation for the online bookstore. Rather than building the software from the ground up, the system utilizes a pre-existing, cloud-hosted platform specifically designed for retail operations.

This approach natively supports all the core user tasks identified in this specification out-of-the-box. Functionalities such as browsing the book catalogue, managing shopping carts, securely processing payments, and generating sales statistics are built-in features of the platform. Staff can also easily manage packaging, shipments, and the book catalogue through an intuitive, pre-built administrative dashboard.

For a local business like Favourite Books looking to quickly transition to an Australia-wide online market, a COTS platform offers a highly streamlined path to deployment. It removes the technical burden of managing underlying infrastructure, servers, and security compliance (such as PCI DSS for payments), as these are entirely handled by the vendor. This allows the store owners to focus purely on business operations and order fulfillment rather than software maintenance.

A major strength of this architecture is its rapid deployment speed, high inherent reliability, and built-in scalability. Because the platform serves thousands of businesses, it is optimized to handle high traffic and provides a polished, responsive user interface from day one.

However, this solution introduces significant limitations regarding deep customizability and control. The business is constrained to the platform's specific ecosystem, relying on available themes and third-party plugins to extend functionality. If Favourite Books requires a highly unique workflow in the future, it may be impossible or prohibitively expensive to implement. Furthermore, this approach incurs ongoing recurring subscription fees, potential transaction-based costs, and creates "vendor lock-in," making it difficult to migrate the store to a custom architecture later.

Overall, a COTS platform like Shopify is an excellent, low-risk solution for Favourite Books to quickly and reliably establish its online presence, provided the business is willing to accept the ongoing subscription costs and predefined architectural constraints.

Solution 3 Pros and Cons Table:

Pro/Con	Details
Pros – Deployment Speed	Fast to set up and launch compared to custom development, allowing the store to go to market quickly.
Pros – Maintenance	Infrastructure, security, and updates are managed by the platform provider, requiring no internal technical staff.
Pros – Built-in Features	Natively supports essential e-commerce tasks like payment processing, cart management, and inventory tracking out-of-the-box.
Pros – Reliability	High uptime and automatic scalability handled by the vendor's enterprise-grade infrastructure.

Cons – Customization	Limited flexibility. The system is strictly bound by the platform's available themes, plugins, and core logic.
Cons – Ongoing Costs	Requires recurring subscription fees and often incurs additional transaction fees or costs for premium plugins.
Cons – Vendor Lock-in	Difficult and time-consuming to migrate business data and operations away from the platform in the future.
Cons – Data Ownership	Less direct control over how the underlying database is structured and how customer data is technically stored.